

CSE thesis SVM P3

January 24, 2026

```
[4]: """
Reusable KNN & SVM Pipeline for Microfinance Impact Analysis
-----
Matches the output style of the Random Forest script (Graphs, Validations,
↳Excel).

Usage:
    run_comprehensive_model_analysis(target_bf="BF1", model_type="svm")
    run_comprehensive_model_analysis(target_bf="BF3", model_type="knn")
"""

# =====
# 0) IMPORTS
# =====
import os, io, re, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold,
↳cross_validate
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score, roc_curve,
    precision_recall_curve, average_precision_score,
    accuracy_score, precision_score, recall_score, f1_score
)
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import joblib

RANDOM_STATE = 42
```

```

# =====
# 1) DATA LOADING
# =====
def load_excel_from_colab():
    """Upload Excel in Google Colab."""
    try:
        from google.colab import files
        print(" Please select the Excel file with your encoded dataset...")
        uploaded = files.upload()
        path = list(uploaded.keys())[0]
        df = pd.read_excel(path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        raise RuntimeError("Colab upload failed.") from e

def load_excel_locally():
    """Upload Excel when running locally."""
    try:
        import tkinter as tk
        from tkinter import filedialog
        print(" Please select the Excel file...")
        root = tk.Tk()
        root.withdraw()
        file_path = filedialog.askopenfilename(
            title="Select Excel file",
            filetypes=[("Excel files", "*.xlsx *.xls")]
        )
        if not file_path:
            raise ValueError("No file selected")
        df = pd.read_excel(file_path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        print(f" File dialog failed: {e}")
        file_path = input(" Enter full path to your Excel file: ")
        df = pd.read_excel(file_path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df

# =====
# 2) TARGET SELECTION
# =====
def detect_bf_target(df: pd.DataFrame, target_bf: str = "BF1") -> str:
    m = re.match(r"^\\s*BF\\s*(\\d+)\\s*$", str(target_bf), flags=re.I)
    if not m:

```

```

        raise ValueError(f"target_bf must look like 'BF1', 'BF3', etc. Got:␣
↪{target_bf}")

k = m.group(1)
aliases = [f"BF{k}", f"bf{k}", f"BF_{k}", f"bf_{k}"]

for a in aliases:
    if a in df.columns:
        print(f" Found target column: {a}")
        return a

pat = re.compile(rf"\bbf\s*[_-]?\s*{k}\b", flags=re.I)
for col in df.columns:
    if pat.search(str(col)):
        print(f" Found target column (regex match): {col}")
        return col

print(f" {target_bf} column not found automatically.")
print(" Available columns:", df.columns.tolist())
user_col = input(f" Enter exact column name for {target_bf}: ").strip()
return user_col

def preprocess_bf_target(y_series: pd.Series, target_label: str):
    y_numeric = pd.to_numeric(y_series, errors="coerce")
    valid_mask = y_numeric.notna() & np.isfinite(y_numeric)
    y_clean = y_numeric[valid_mask]

    if len(y_clean) == 0:
        raise ValueError(f" No valid values for {target_label}")

    print(f"\n TARGET ANALYSIS: {target_label}")
    print("-" * 40)
    print("Value distribution:\n", y_clean.value_counts().sort_index())

    y_binary = y_clean.astype(int)
    class_balance = y_binary.value_counts(normalize=True).round(3)
    print(f" Final class balance: {class_balance.to_dict()}")

    return y_binary, valid_mask

# =====
# 3) FEATURE ENGINEERING
# =====
def build_feature_matrix(df_clean: pd.DataFrame, target_col: str):
    print("\n FEATURE ENGINEERING")
    print("-" * 40)

```

```

X = df_clean.drop(columns=[target_col], errors="ignore")
numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()

imputer = SimpleImputer(strategy="median")
X_imputed = imputer.fit_transform(X[numeric_cols])
X_imputed = pd.DataFrame(X_imputed, columns=numeric_cols, index=X.index)

non_constant_cols = [c for c in X_imputed.columns if X_imputed[c].nunique()
↳ > 1]
X_final = X_imputed[non_constant_cols]

print(f" Final feature matrix: {X_final.shape} (rows, cols)")
return X_final, non_constant_cols

# =====
# 4) MODEL DEFINITIONS
# =====
def get_model_pipeline(model_type, n_samples=None):
    """Returns a pipeline with Scaler + Classifier based on model_type."""

    # Preprocessing (Scaling is MANDATORY for SVM/KNN)
    preprocessor = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])

    if model_type == "knn":
        # Dynamic K: Sqrt(N) is a good rule of thumb, but keep it odd
        k = int(np.sqrt(n_samples)) if n_samples else 5
        if k % 2 == 0: k += 1
        print(f" Configured KNN with n_neighbors={k}")

        clf = KNeighborsClassifier(n_neighbors=k, weights='distance', n_jobs=-1)

    elif model_type == "svm":
        print(" Configured SVM (RBF Kernel, C=1.0, Probability=True)")
        clf = SVC(
            kernel='rbf',
            C=1.0,
            class_weight='balanced',
            probability=True, # Needed for ROC Curves
            random_state=RANDOM_STATE
        )
    else:
        raise ValueError("model_type must be 'knn' or 'svm'")

    return Pipeline([("pre", preprocessor), ("clf", clf)])

```

```

# =====
# 5) CROSS-VALIDATION
# =====
def perform_cross_validation(X, y, model_type, cv_folds=10):
    print(f"\n PERFORMING {cv_folds}-FOLD CROSS-VALIDATION ({model_type.
    ↪upper()})")
    print("=" * 50)

    pipe = get_model_pipeline(model_type, n_samples=len(X))

    scoring = {
        "accuracy": "accuracy", "precision": "precision", "recall": "recall",
        "f1": "f1", "roc_auc": "roc_auc", "average_precision": ↪
    ↪"average_precision"
    }

    cv_results = cross_validate(
        pipe, X, y,
        cv=StratifiedKFold(n_splits=cv_folds, shuffle=True, ↪
    ↪random_state=RANDOM_STATE),
        scoring=scoring,
        return_train_score=True,
        n_jobs=-1
    )

    print(" CROSS-VALIDATION RESULTS (Mean ± Std):")
    print("-" * 40)

    metrics_summary = {}
    for metric in scoring.keys():
        test_scores = cv_results[f"test_{metric}"]
        train_scores = cv_results[f"train_{metric}"]

        metrics_summary[metric] = {
            "test_mean": float(np.mean(test_scores)),
            "test_std": float(np.std(test_scores)),
            "train_mean": float(np.mean(train_scores))
        }
        print(f"{metric.upper():<15}: {np.mean(test_scores):.4f} ± {np.
    ↪std(test_scores):.4f}")

    return metrics_summary, cv_results

# =====
# 6) TRAINING
# =====

```

```

def create_validation_set(X, y, validation_size=0.2):
    print(f"\n CREATING SPLITS (Val={validation_size})")
    X_train_test, X_val, y_train_test, y_val = train_test_split(
        X, y, test_size=validation_size, stratify=y, random_state=RANDOM_STATE
    )
    X_train, X_test, y_train, y_test = train_test_split(
        X_train_test, y_train_test, test_size=0.25, stratify=y_train_test,
        random_state=RANDOM_STATE
    )
    return X_train, X_test, X_val, y_train, y_test, y_val

def train_model_comprehensive(X, y, model_type, use_validation_set=True):
    print(f"\n MODEL TRAINING: {model_type.upper()}")
    print("-" * 40)

    if use_validation_set:
        X_train, X_test, X_val, y_train, y_test, y_val =
        create_validation_set(X, y)
    else:
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.
        25, stratify=y, random_state=RANDOM_STATE)
        X_val, y_val = None, None

    # Get pipeline with correct scaling + classifier
    pipe = get_model_pipeline(model_type, n_samples=len(X_train))

    print(" Fitting model...")
    pipe.fit(X_train, y_train)

    return pipe, X_train, X_test, X_val, y_train, y_test, y_val

# =====
# 7) EVALUATION & VISUALIZATION
# =====
def evaluate_model_metrics(model, X_test, y_test, dataset_name="Test"):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    metrics = {
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred, zero_division=0),
        "recall": recall_score(y_test, y_pred, zero_division=0),
        "f1": f1_score(y_test, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_test, y_proba),
        "pr_auc": average_precision_score(y_test, y_proba)
    }

```

```

print("\n" + "=" * 60)
print(f" {dataset_name.upper()} SET EVALUATION")
print("=" * 60)
for k, v in metrics.items():
    print(f" {k.capitalize():<10}: {v:.4f}")

    print(f"\n {dataset_name} Classification Report:\n",
↪classification_report(y_test, y_pred, digits=3))
    print(f" {dataset_name} Confusion Matrix:\n", confusion_matrix(y_test,
↪y_pred))

    return y_pred, y_proba, metrics

def create_visualizations(y_test, y_proba, model_type, output_dir, prefix,
↪cv_metrics=None):
    os.makedirs(output_dir, exist_ok=True)

    # 1. ROC & PR Curves
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

    fpr, tpr, _ = roc_curve(y_test, y_proba)
    roc_auc = roc_auc_score(y_test, y_proba)
    ax1.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}", linewidth=2, color="C0")
    ax1.plot([0, 1], [0, 1], "k--", alpha=0.5)
    ax1.set_title(f"ROC Curve ({model_type.upper()})")
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    prec, rec, _ = precision_recall_curve(y_test, y_proba)
    pr_auc = average_precision_score(y_test, y_proba)
    ax2.plot(rec, prec, label=f"AP = {pr_auc:.3f}", linewidth=2, color="C1")
    ax2.set_title(f"Precision-Recall Curve ({model_type.upper()})")
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(f"{output_dir}/{prefix}_roc_pr_curves.png", dpi=200)
    plt.show()

    # 2. Confusion Matrix
    y_pred = (y_proba >= 0.5).astype(int)
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Pred 0",
↪"Pred 1"], yticklabels=["True 0", "True 1"])
    plt.title(f"Confusion Matrix ({model_type.upper()})")
    plt.savefig(f"{output_dir}/{prefix}_confusion_matrix.png", dpi=200)

```

```

plt.show()

# 3. CV Comparison
if cv_metrics:
    names = list(cv_metrics.keys())
    means = [cv_metrics[m]["test_mean"] for m in names]
    stds = [cv_metrics[m]["test_std"] for m in names]

    plt.figure(figsize=(10, 5))
    bars = plt.bar(names, means, yerr=stds, capsize=5, alpha=0.7)
    for i, b in enumerate(bars): b.set_color(f"C{i}")
    plt.title(f"Cross-Validation Performance ({model_type.upper()})")
    plt.ylim(0, 1)
    plt.grid(axis='y', alpha=0.3)
    plt.savefig(f"{output_dir}/{prefix}_cv_results.png", dpi=200)
    plt.show()

# =====
# 8) SAVING RESULTS
# =====
def save_results(model, metrics, df_clean, target_col, feature_names,
    ↪output_dir, prefix, cv_metrics=None):
    print(f"\n SAVING OUTPUTS to {output_dir}/ ...")

    # 1. Metrics Text File
    with open(f"{output_dir}/{prefix}_metrics.txt", "w") as f:
        f.write(f"MODEL RESULTS: {prefix.upper()}\n{' '*40}\n")
        f.write("TEST SET METRICS:\n")
        for k, v in metrics.items(): f.write(f"{k}: {v:.4f}\n")

        if cv_metrics:
            f.write("\nCROSS-VALIDATION:\n")
            for k, v in cv_metrics.items():
                f.write(f"{k}: {v['test_mean']:.4f} ± {v['test_std']:.4f}\n")

    # 2. Excel Predictions
    X_full = df_clean.drop(columns=[target_col], errors="ignore").
    ↪reindex(columns=feature_names)
    preds = model.predict_proba(X_full)[: , 1]

    out_df = df_clean.copy()
    out_df[f"{prefix}_Prob"] = preds
    out_df[f"{prefix}_Pred"] = (preds >= 0.5).astype(int)
    out_df.to_excel(f"{output_dir}/{prefix}_predictions.xlsx", index=False)

    # 3. Model
    joblib.dump(model, f"{output_dir}/{prefix}_model.joblib")

```



```

print(" All files saved successfully.")

# =====
# 9) MAIN PIPELINE
# =====
def run_comprehensive_model_analysis(target_bf="BF1", model_type="svm",
    ↪df=None):
    """
    Run pipeline for SVM or KNN.
    model_type: 'svm' or 'knn'
    """
    try:
        model_type = model_type.lower()
        prefix = f"{target_bf}_{model_type}"
        output_dir = f"{prefix}_outputs"

        print(f" STARTING ANALYSIS: {model_type.upper()} on {target_bf}")

        if df is None:
            try: df = load_excel_from_colab()
            except: df = load_excel_locally()

        # Target Setup
        target_col = detect_bf_target(df, target_bf)
        y, valid_mask = preprocess_bf_target(df[target_col], target_col)
        df_clean = df.loc[valid_mask].copy()

        # Features
        X_final, feature_names = build_feature_matrix(df_clean, target_col)

        # 1. CV
        cv_metrics, _ = perform_cross_validation(X_final, y, model_type,
    ↪cv_folds=10)

        # 2. Train
        model, X_train, X_test, X_val, y_train, y_test, y_val =
    ↪train_model_comprehensive(
            X_final, y, model_type, use_validation_set=True
        )

        # 3. Eval
        _, y_proba, test_metrics = evaluate_model_metrics(model, X_test,
    ↪y_test, "Test")

        # 4. Viz
        create_visualizations(y_test, y_proba, model_type, output_dir, prefix,
    ↪cv_metrics)

```

```

    # 5. Save
    save_results(model, test_metrics, df_clean, target_col, feature_names,
    ↪output_dir, prefix, cv_metrics)

    print("\n DONE.")
    return model

except Exception as e:
    print(f"\n Error: {e}")
    return None

# ===== USAGE =====
if __name__ == "__main__":
    # Example 1: Run SVM on BF6 (Default)
    run_comprehensive_model_analysis(target_bf="BF6", model_type="svm")

    # Example 2: Run KNN (Uncomment to use)
    # run_comprehensive_model_analysis(target_bf="BF6", model_type="knn")

```

STARTING ANALYSIS: SVM on BF6

Please select the Excel file with your encoded dataset...

<IPython.core.display.HTML object>

Saving rf_dataset_encoded_dataset.xlsx to rf_dataset_encoded_dataset (1).xlsx

Loaded dataset: (1002, 85) (rows, cols)

BF6 column not found automatically.

Available columns: ['D', 'AY', 'AZ', 'BA', 'BB', 'BC', 'BD', 'BE_01', 'BE_02', 'BE_03', 'BE_04', 'BE_05', 'BE_06', 'BE_07', 'BE_08', 'BE_09', 'BE_10', 'BF', 'BG_01', 'BG_02', 'BG_03', 'BG_04', 'BG_05', 'BG_06', 'BH', 'BI', 'BN', 'BO', 'BP', 'BQ', 'BR', 'BS', 'BT', 'BU', 'BV', 'BW_01', 'BW_02', 'BW_03', 'BW_04', 'BW_05', 'BX', 'BY', 'CB', 'CD', 'CE', 'CF', 'CG', 'CH', 'CI', 'CK', 'CL', 'CM_01', 'CM_02', 'CM_03', 'CM_04', 'CM_05', 'CM_06', 'CN', 'CO_01', 'CO_02', 'CO_03', 'CO_04', 'CO_05', 'CO_06', 'CO_07', 'CO_08', 'CO_09', 'CP', 'CR_01', 'CR_02', 'CR_03', 'CR_04', 'CS', 'CT_01', 'CT_02', 'CT_03', 'CT_04', 'CT_05', 'CT_06', 'CU', 'Occupation', 'Financial Behavior', 'Family', 'Lifestyle', 'Digital Literacy']

Enter exact column name for BF6: BG_01

TARGET ANALYSIS: BG_01

Value distribution:

BG_01

0 533

1 469

Name: count, dtype: int64

Final class balance: {0: 0.532, 1: 0.468}

FEATURE ENGINEERING

Final feature matrix: (1002, 84) (rows, cols)

PERFORMING 10-FOLD CROSS-VALIDATION (SVM)

=====
Configured SVM (RBF Kernel, C=1.0, Probability=True)
CROSS-VALIDATION RESULTS (Mean $\pm$ Std):

ACCURACY : 0.8013 $\pm$ 0.0414
PRECISION : 0.7618 $\pm$ 0.0468
RECALL : 0.8424 $\pm$ 0.0562
F1 : 0.7988 $\pm$ 0.0399
ROC_AUC : 0.8959 $\pm$ 0.0270
AVERAGE_PRECISION: 0.8845 $\pm$ 0.0279

MODEL TRAINING: SVM

CREATING SPLITS (Val=0.2)

Configured SVM (RBF Kernel, C=1.0, Probability=True)
Fitting model...

TEST SET EVALUATION

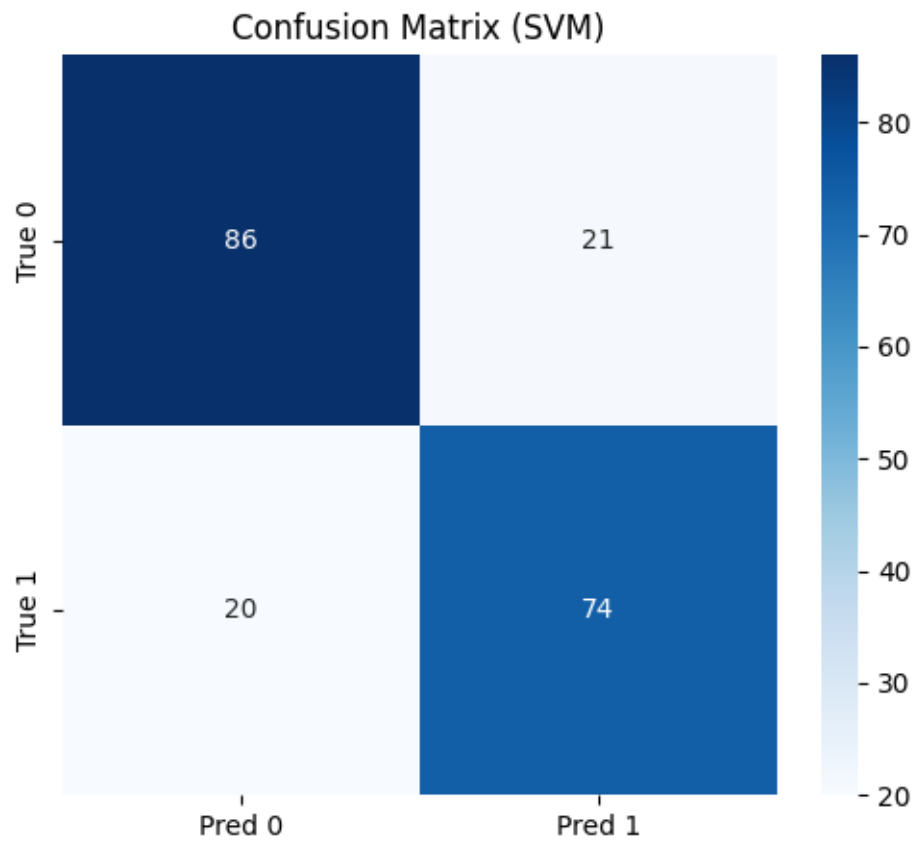
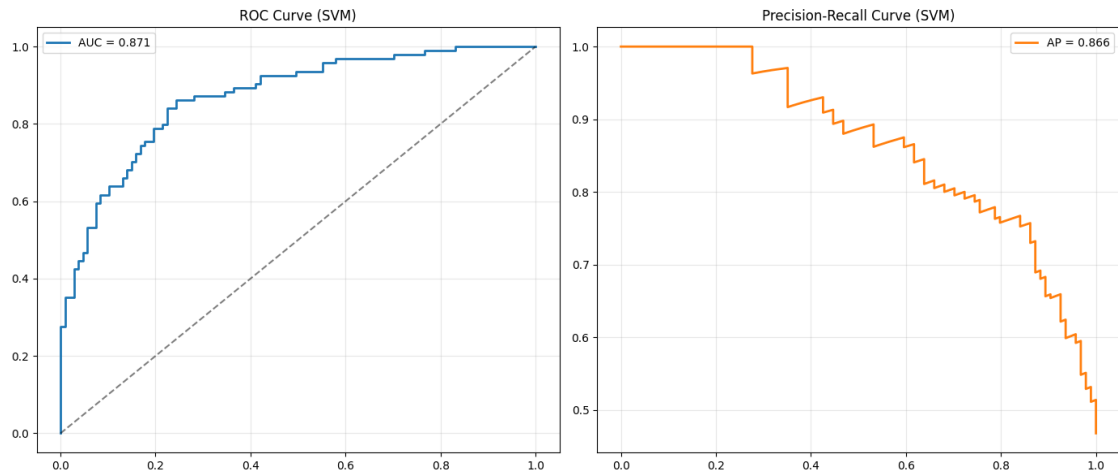
=====
Accuracy : 0.8010
Precision : 0.7500
Recall : 0.8617
F1 : 0.8020
Roc_auc : 0.8706
Pr_auc : 0.8664

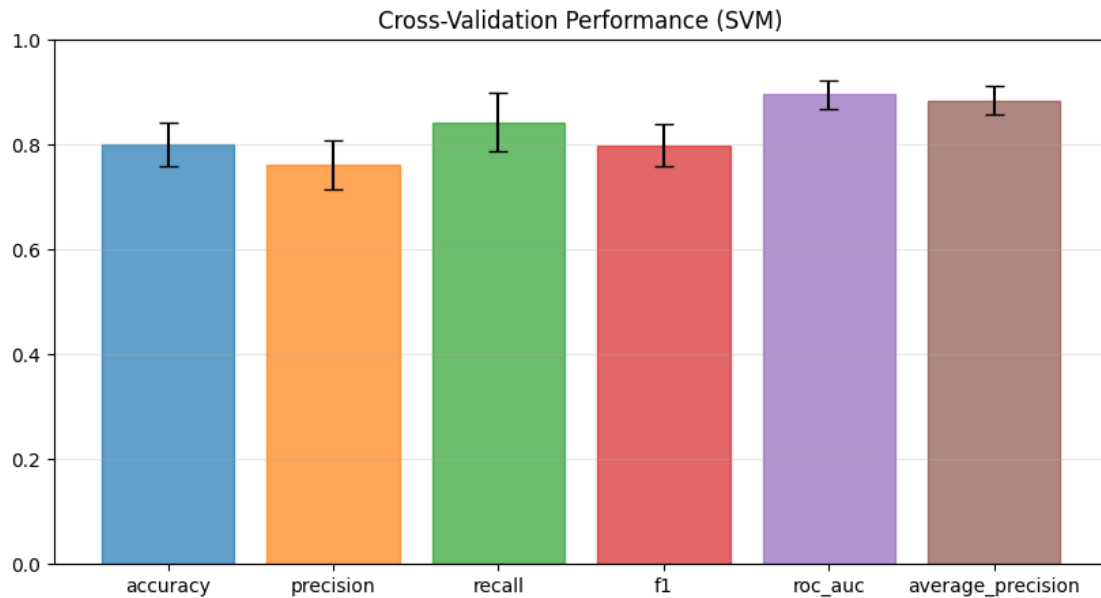
Test Classification Report:

	precision	recall	f1-score	support
0	0.860	0.748	0.800	107
1	0.750	0.862	0.802	94
accuracy			0.801	201
macro avg	0.805	0.805	0.801	201
weighted avg	0.809	0.801	0.801	201

Test Confusion Matrix:

[[80 27]
[13 81]]





SAVING OUTPUTS to BF6_svm_outputs/ ...
All files saved successfully.

DONE.

```
[5]: """
Reusable KNN & SVM Pipeline for Microfinance Impact Analysis
-----
Matches the output style of the Random Forest script (Graphs, Validations,
↳Excel).

Usage:
    run_comprehensive_model_analysis(target_bf="BF1", model_type="svm")
    run_comprehensive_model_analysis(target_bf="BF3", model_type="knn")
"""

# =====
# 0) IMPORTS
# =====
import os, io, re, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```

from sklearn.model_selection import train_test_split, StratifiedKFold,
    ↪cross_validate
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score, roc_curve,
    precision_recall_curve, average_precision_score,
    accuracy_score, precision_score, recall_score, f1_score
)
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import joblib

RANDOM_STATE = 42

# =====
# 1) DATA LOADING
# =====
def load_excel_from_colab():
    """Upload Excel in Google Colab."""
    try:
        from google.colab import files
        print(" Please select the Excel file with your encoded dataset...")
        uploaded = files.upload()
        path = list(uploaded.keys())[0]
        df = pd.read_excel(path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        raise RuntimeError("Colab upload failed.") from e

def load_excel_locally():
    """Upload Excel when running locally."""
    try:
        import tkinter as tk
        from tkinter import filedialog
        print(" Please select the Excel file...")
        root = tk.Tk()
        root.withdraw()
        file_path = filedialog.askopenfilename(
            title="Select Excel file",
            filetypes=[("Excel files", "*.xlsx *.xls")]
        )
        if not file_path:
            raise ValueError("No file selected")
    
```

```

    df = pd.read_excel(file_path)
    print(f" Loaded dataset: {df.shape} (rows, cols)")
    return df
except Exception as e:
    print(f" File dialog failed: {e}")
    file_path = input(" Enter full path to your Excel file: ")
    df = pd.read_excel(file_path)
    print(f" Loaded dataset: {df.shape} (rows, cols)")
    return df

# =====
# 2) TARGET SELECTION
# =====
def detect_bf_target(df: pd.DataFrame, target_bf: str = "BF1") -> str:
    m = re.match(r"^s*BF\s*(\d+)\s*$", str(target_bf), flags=re.I)
    if not m:
        raise ValueError(f"target_bf must look like 'BF1', 'BF3', etc. Got: {target_bf}")

    k = m.group(1)
    aliases = [f"BF{k}", f"bf{k}", f"BF_{k}", f"bf_{k}"]

    for a in aliases:
        if a in df.columns:
            print(f" Found target column: {a}")
            return a

    pat = re.compile(rf"\bbf\s*[_-]?s*{k}\b", flags=re.I)
    for col in df.columns:
        if pat.search(str(col)):
            print(f" Found target column (regex match): {col}")
            return col

    print(f" {target_bf} column not found automatically.")
    print(" Available columns:", df.columns.tolist())
    user_col = input(f" Enter exact column name for {target_bf}: ").strip()
    return user_col

def preprocess_bf_target(y_series: pd.Series, target_label: str):
    y_numeric = pd.to_numeric(y_series, errors="coerce")
    valid_mask = y_numeric.notna() & np.isfinite(y_numeric)
    y_clean = y_numeric[valid_mask]

    if len(y_clean) == 0:
        raise ValueError(f" No valid values for {target_label}")

    print(f"\n TARGET ANALYSIS: {target_label}")

```

```

print("-" * 40)
print("Value distribution:\n", y_clean.value_counts().sort_index())

y_binary = y_clean.astype(int)
class_balance = y_binary.value_counts(normalize=True).round(3)
print(f" Final class balance: {class_balance.to_dict()}")

return y_binary, valid_mask

# =====
# 3) FEATURE ENGINEERING
# =====
def build_feature_matrix(df_clean: pd.DataFrame, target_col: str):
    print("\n FEATURE ENGINEERING")
    print("-" * 40)

    X = df_clean.drop(columns=[target_col], errors="ignore")
    numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()

    imputer = SimpleImputer(strategy="median")
    X_imputed = imputer.fit_transform(X[numeric_cols])
    X_imputed = pd.DataFrame(X_imputed, columns=numeric_cols, index=X.index)

    non_constant_cols = [c for c in X_imputed.columns if X_imputed[c].nunique()
↪ > 1]
    X_final = X_imputed[non_constant_cols]

    print(f" Final feature matrix: {X_final.shape} (rows, cols)")
    return X_final, non_constant_cols

# =====
# 4) MODEL DEFINITIONS
# =====
def get_model_pipeline(model_type, n_samples=None):
    """Returns a pipeline with Scaler + Classifier based on model_type."""

    # Preprocessing (Scaling is MANDATORY for SVM/KNN)
    preprocessor = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])

    if model_type == "knn":
        # Dynamic K: Sqrt(N) is a good rule of thumb, but keep it odd
        k = int(np.sqrt(n_samples)) if n_samples else 5
        if k % 2 == 0: k += 1
        print(f" Configured KNN with n_neighbors={k}")

```



```

        clf = KNeighborsClassifier(n_neighbors=k, weights='distance', n_jobs=-1)

    elif model_type == "svm":
        print(" Configured SVM (RBF Kernel, C=1.0, Probability=True)")
        clf = SVC(
            kernel='rbf',
            C=1.0,
            class_weight='balanced',
            probability=True, # Needed for ROC Curves
            random_state=RANDOM_STATE
        )
    else:
        raise ValueError("model_type must be 'knn' or 'svm'")

    return Pipeline([("pre", preprocessor), ("clf", clf)])

# =====
# 5) CROSS-VALIDATION
# =====
def perform_cross_validation(X, y, model_type, cv_folds=10):
    print(f"\n PERFORMING {cv_folds}-FOLD CROSS-VALIDATION ({model_type}.
    ↪upper()))")
    print("=" * 50)

    pipe = get_model_pipeline(model_type, n_samples=len(X))

    scoring = {
        "accuracy": "accuracy", "precision": "precision", "recall": "recall",
        "f1": "f1", "roc_auc": "roc_auc", "average_precision": ↪
    ↪"average_precision"
    }

    cv_results = cross_validate(
        pipe, X, y,
        cv=StratifiedKFold(n_splits=cv_folds, shuffle=True, ↪
    ↪random_state=RANDOM_STATE),
        scoring=scoring,
        return_train_score=True,
        n_jobs=-1
    )

    print(" CROSS-VALIDATION RESULTS (Mean ± Std):")
    print("-" * 40)

    metrics_summary = {}
    for metric in scoring.keys():

```

```

test_scores = cv_results[f"test_{metric}"]
train_scores = cv_results[f"train_{metric}"]

metrics_summary[metric] = {
    "test_mean": float(np.mean(test_scores)),
    "test_std": float(np.std(test_scores)),
    "train_mean": float(np.mean(train_scores))
}
print(f"{metric.upper():<15}: {np.mean(test_scores):.4f} ± {np.
↪std(test_scores):.4f}")

return metrics_summary, cv_results

# =====
# 6) TRAINING
# =====
def create_validation_set(X, y, validation_size=0.2):
    print(f"\n CREATING SPLITS (Val={validation_size})")
    X_train_test, X_val, y_train_test, y_val = train_test_split(
        X, y, test_size=validation_size, stratify=y, random_state=RANDOM_STATE
    )
    X_train, X_test, y_train, y_test = train_test_split(
        X_train_test, y_train_test, test_size=0.25, stratify=y_train_test, ↪
↪random_state=RANDOM_STATE
    )
    return X_train, X_test, X_val, y_train, y_test, y_val

def train_model_comprehensive(X, y, model_type, use_validation_set=True):
    print(f"\n MODEL TRAINING: {model_type.upper()}")
    print("-" * 40)

    if use_validation_set:
        X_train, X_test, X_val, y_train, y_test, y_val = ↪
↪create_validation_set(X, y)
    else:
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.
↪25, stratify=y, random_state=RANDOM_STATE)
        X_val, y_val = None, None

    # Get pipeline with correct scaling + classifier
    pipe = get_model_pipeline(model_type, n_samples=len(X_train))

    print(" Fitting model...")
    pipe.fit(X_train, y_train)

    return pipe, X_train, X_test, X_val, y_train, y_test, y_val

```

```

# =====
# 7) EVALUATION & VISUALIZATION
# =====
def evaluate_model_metrics(model, X_test, y_test, dataset_name="Test"):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    metrics = {
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred, zero_division=0),
        "recall": recall_score(y_test, y_pred, zero_division=0),
        "f1": f1_score(y_test, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_test, y_proba),
        "pr_auc": average_precision_score(y_test, y_proba)
    }

    print("\n" + "=" * 60)
    print(f" {dataset_name.upper()} SET EVALUATION")
    print("=" * 60)
    for k, v in metrics.items():
        print(f" {k.capitalize():<10}: {v:.4f}")

    print(f"\n {dataset_name} Classification Report:\n",
    ↪classification_report(y_test, y_pred, digits=3))
    print(f" {dataset_name} Confusion Matrix:\n", confusion_matrix(y_test,
    ↪y_pred))

    return y_pred, y_proba, metrics

def create_visualizations(y_test, y_proba, model_type, output_dir, prefix,
    ↪cv_metrics=None):
    os.makedirs(output_dir, exist_ok=True)

    # 1. ROC & PR Curves
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

    fpr, tpr, _ = roc_curve(y_test, y_proba)
    roc_auc = roc_auc_score(y_test, y_proba)
    ax1.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}", linewidth=2, color="C0")
    ax1.plot([0, 1], [0, 1], "k--", alpha=0.5)
    ax1.set_title(f"ROC Curve ({model_type.upper()})")
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    prec, rec, _ = precision_recall_curve(y_test, y_proba)
    pr_auc = average_precision_score(y_test, y_proba)
    ax2.plot(rec, prec, label=f"AP = {pr_auc:.3f}", linewidth=2, color="C1")

```

```

ax2.set_title(f"Precision-Recall Curve ({model_type.upper()})")
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_roc_pr_curves.png", dpi=200)
plt.show()

# 2. Confusion Matrix
y_pred = (y_proba >= 0.5).astype(int)
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Pred 0", "Pred 1"], yticklabels=["True 0", "True 1"])
plt.title(f"Confusion Matrix ({model_type.upper()})")
plt.savefig(f"{output_dir}/{prefix}_confusion_matrix.png", dpi=200)
plt.show()

# 3. CV Comparison
if cv_metrics:
    names = list(cv_metrics.keys())
    means = [cv_metrics[m]["test_mean"] for m in names]
    stds = [cv_metrics[m]["test_std"] for m in names]

    plt.figure(figsize=(10, 5))
    bars = plt.bar(names, means, yerr=stds, capsize=5, alpha=0.7)
    for i, b in enumerate(bars): b.set_color(f"C{i}")
    plt.title(f"Cross-Validation Performance ({model_type.upper()})")
    plt.ylim(0, 1)
    plt.grid(axis='y', alpha=0.3)
    plt.savefig(f"{output_dir}/{prefix}_cv_results.png", dpi=200)
    plt.show()

# =====
# 8) SAVING RESULTS
# =====
def save_results(model, metrics, df_clean, target_col, feature_names,
                 output_dir, prefix, cv_metrics=None):
    print(f"\n SAVING OUTPUTS to {output_dir}/ ...")

    # 1. Metrics Text File
    with open(f"{output_dir}/{prefix}_metrics.txt", "w") as f:
        f.write(f"MODEL RESULTS: {prefix.upper()}\n{'='*40}\n")
        f.write("TEST SET METRICS:\n")
        for k, v in metrics.items(): f.write(f"{k}: {v:.4f}\n")

    if cv_metrics:

```

```

        f.write("\nCROSS-VALIDATION:\n")
        for k, v in cv_metrics.items():
            f.write(f"{k}: {v['test_mean']:.4f} ± {v['test_std']:.4f}\n")

# 2. Excel Predictions
X_full = df_clean.drop(columns=[target_col], errors="ignore").
↳reindex(columns=feature_names)
preds = model.predict_proba(X_full)[: , 1]

out_df = df_clean.copy()
out_df[f"{prefix}_Prob"] = preds
out_df[f"{prefix}_Pred"] = (preds >= 0.5).astype(int)
out_df.to_excel(f"{output_dir}/{prefix}_predictions.xlsx", index=False)

# 3. Model
joblib.dump(model, f"{output_dir}/{prefix}_model.joblib")
print(" All files saved successfully.")

# =====
# 9) MAIN PIPELINE
# =====
def run_comprehensive_model_analysis(target_bf="BF1", model_type="svm",
↳df=None):
    """
    Run pipeline for SVM or KNN.
    model_type: 'svm' or 'knn'
    """
    try:
        model_type = model_type.lower()
        prefix = f"{target_bf}_{model_type}"
        output_dir = f"{prefix}_outputs"

        print(f" STARTING ANALYSIS: {model_type.upper()} on {target_bf}")

        if df is None:
            try: df = load_excel_from_colab()
            except: df = load_excel_locally()

# Target Setup
target_col = detect_bf_target(df, target_bf)
y, valid_mask = preprocess_bf_target(df[target_col], target_col)
df_clean = df.loc[valid_mask].copy()

# Features
X_final, feature_names = build_feature_matrix(df_clean, target_col)

# 1. CV

```

```

        cv_metrics, _ = perform_cross_validation(X_final, y, model_type,
↪cv_folds=10)

        # 2. Train
        model, X_train, X_test, X_val, y_train, y_test, y_val =
↪train_model_comprehensive(
            X_final, y, model_type, use_validation_set=True
        )

        # 3. Eval
        _, y_proba, test_metrics = evaluate_model_metrics(model, X_test,
↪y_test, "Test")

        # 4. Viz
        create_visualizations(y_test, y_proba, model_type, output_dir, prefix,
↪cv_metrics)

        # 5. Save
        save_results(model, test_metrics, df_clean, target_col, feature_names,
↪output_dir, prefix, cv_metrics)

        print("\n DONE.")
        return model

    except Exception as e:
        print(f"\n Error: {e}")
        return None

# ===== USAGE =====
if __name__ == "__main__":
    # Example 1: Run SVM on BF6 (Default)
    run_comprehensive_model_analysis(target_bf="BF6", model_type="svm")

    # Example 2: Run KNN (Uncomment to use)
    # run_comprehensive_model_analysis(target_bf="BF6", model_type="knn")

```

STARTING ANALYSIS: SVM on BF6

Please select the Excel file with your encoded dataset...

<IPython.core.display.HTML object>

Saving rf_dataset_encoded_dataset.xlsx to rf_dataset_encoded_dataset (2).xlsx

Loaded dataset: (1002, 85) (rows, cols)

BF6 column not found automatically.

Available columns: ['D', 'AY', 'AZ', 'BA', 'BB', 'BC', 'BD', 'BE_01', 'BE_02', 'BE_03', 'BE_04', 'BE_05', 'BE_06', 'BE_07', 'BE_08', 'BE_09', 'BE_10', 'BF', 'BG_01', 'BG_02', 'BG_03', 'BG_04', 'BG_05', 'BG_06', 'BH', 'BI', 'BN', 'BO', 'BP', 'BQ', 'BR', 'BS', 'BT', 'BU', 'BV', 'BW_01', 'BW_02', 'BW_03', 'BW_04',

```
'BW_05', 'BX', 'BY', 'CB', 'CD', 'CE', 'CF', 'CG', 'CH', 'CI', 'CK', 'CL',  
'CM_01', 'CM_02', 'CM_03', 'CM_04', 'CM_05', 'CM_06', 'CN', 'CO_01', 'CO_02',  
'CO_03', 'CO_04', 'CO_05', 'CO_06', 'CO_07', 'CO_08', 'CO_09', 'CP', 'CR_01',  
'CR_02', 'CR_03', 'CR_04', 'CS', 'CT_01', 'CT_02', 'CT_03', 'CT_04', 'CT_05',  
'CT_06', 'CU', 'Occupation', 'Financial Behavior', 'Family', 'Lifestyle',  
'Digital Literacy']
```

```
Enter exact column name for BF6: BG_03
```

```
TARGET ANALYSIS: BG_03
```

```
-----  
Value distribution:
```

```
  BG_03
```

```
0      590
```

```
1      412
```

```
Name: count, dtype: int64
```

```
Final class balance: {0: 0.589, 1: 0.411}
```

```
FEATURE ENGINEERING
```

```
-----  
Final feature matrix: (1002, 84) (rows, cols)
```

```
PERFORMING 10-FOLD CROSS-VALIDATION (SVM)
```

```
=====
```

```
Configured SVM (RBF Kernel, C=1.0, Probability=True)
```

```
CROSS-VALIDATION RESULTS (Mean ± Std):
```

```
-----  
ACCURACY           : 0.8194 ± 0.0285  
PRECISION          : 0.7600 ± 0.0372  
RECALL             : 0.8226 ± 0.0649  
F1                 : 0.7885 ± 0.0385  
ROC_AUC            : 0.9022 ± 0.0343  
AVERAGE_PRECISION: 0.8689 ± 0.0385
```

```
MODEL TRAINING: SVM
```

```
-----  
CREATING SPLITS (Val=0.2)
```

```
Configured SVM (RBF Kernel, C=1.0, Probability=True)
```

```
Fitting model...
```

```
=====
```

```
TEST SET EVALUATION
```

```
=====
```

Accuracy	: 0.7662
Precision	: 0.7195
Recall	: 0.7108
F1	: 0.7152
Roc_auc	: 0.8649

Pr_auc : 0.8176

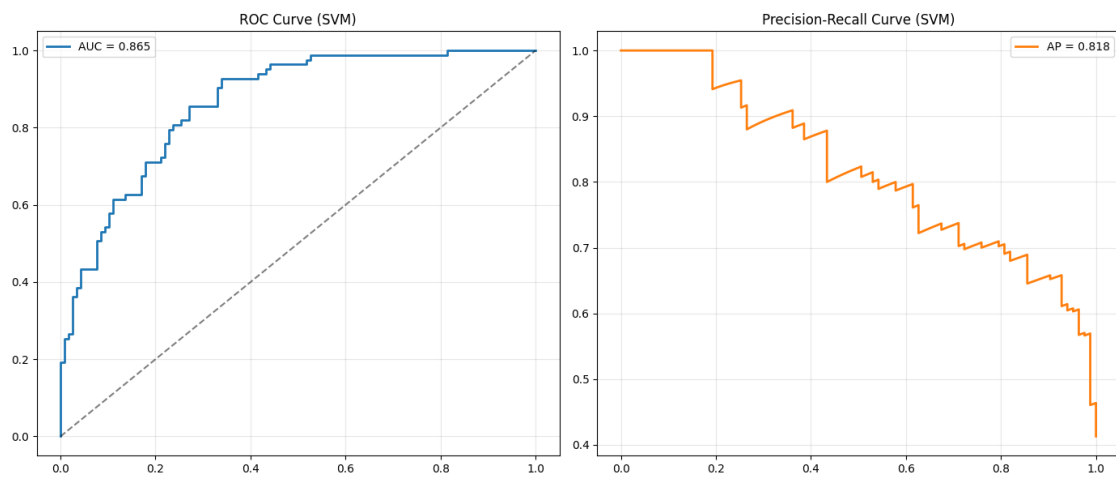
Test Classification Report:

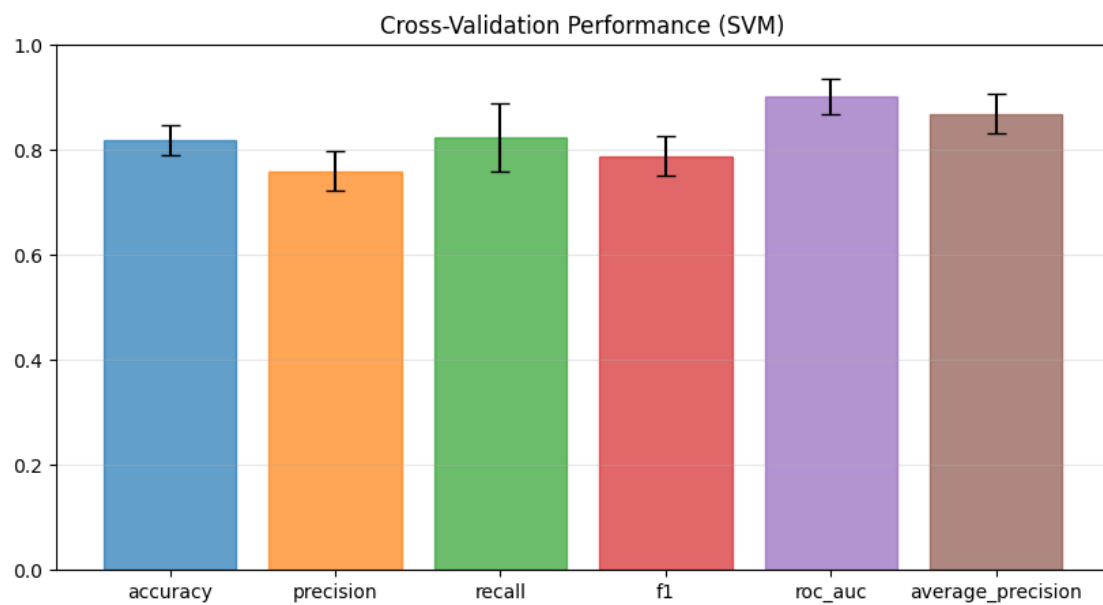
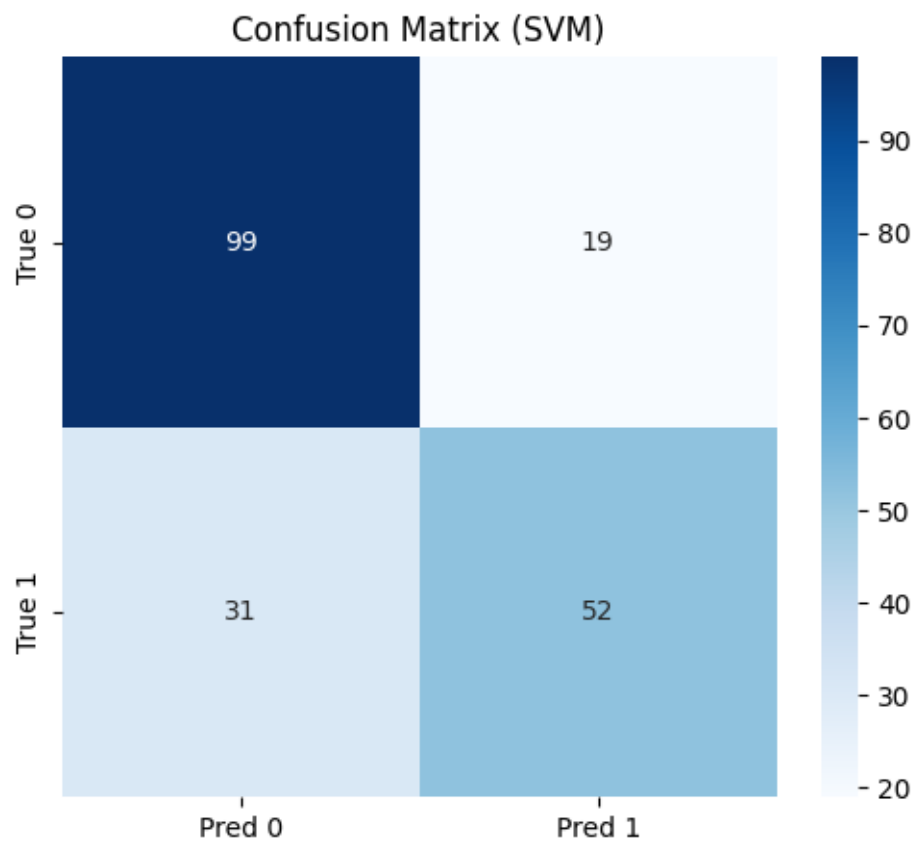
	precision	recall	f1-score	support
0	0.798	0.805	0.802	118
1	0.720	0.711	0.715	83
accuracy			0.766	201
macro avg	0.759	0.758	0.758	201
weighted avg	0.766	0.766	0.766	201

Test Confusion Matrix:

[[95 23]

[24 59]]





SAVING OUTPUTS to BF6_svm_outputs/ ...
All files saved successfully.

DONE.

```
[6]: """
Reusable KNN & SVM Pipeline for Microfinance Impact Analysis
-----
Matches the output style of the Random Forest script (Graphs, Validations,
↳Excel).

Usage:
    run_comprehensive_model_analysis(target_bf="BF1", model_type="svm")
    run_comprehensive_model_analysis(target_bf="BF3", model_type="knn")
"""

# =====
# 0) IMPORTS
# =====
import os, io, re, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold,
↳cross_validate
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score, roc_curve,
    precision_recall_curve, average_precision_score,
    accuracy_score, precision_score, recall_score, f1_score
)
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import joblib

RANDOM_STATE = 42

# =====
# 1) DATA LOADING
# =====
```

```

def load_excel_from_colab():
    """Upload Excel in Google Colab."""
    try:
        from google.colab import files
        print(" Please select the Excel file with your encoded dataset...")
        uploaded = files.upload()
        path = list(uploaded.keys())[0]
        df = pd.read_excel(path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        raise RuntimeError("Colab upload failed.") from e

def load_excel_locally():
    """Upload Excel when running locally."""
    try:
        import tkinter as tk
        from tkinter import filedialog
        print(" Please select the Excel file...")
        root = tk.Tk()
        root.withdraw()
        file_path = filedialog.askopenfilename(
            title="Select Excel file",
            filetypes=[("Excel files", "*.xlsx *.xls")]
        )
        if not file_path:
            raise ValueError("No file selected")
        df = pd.read_excel(file_path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        print(f" File dialog failed: {e}")
        file_path = input(" Enter full path to your Excel file: ")
        df = pd.read_excel(file_path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df

# =====
# 2) TARGET SELECTION
# =====
def detect_bf_target(df: pd.DataFrame, target_bf: str = "BF1") -> str:
    m = re.match(r"^\s*BF\s*(\d+)\s*$", str(target_bf), flags=re.I)
    if not m:
        raise ValueError(f"target_bf must look like 'BF1', 'BF3', etc. Got: {target_bf}")

    k = m.group(1)

```

```

aliases = [f"BF{k}", f"bf{k}", f"BF_{k}", f"bf_{k}"]

for a in aliases:
    if a in df.columns:
        print(f" Found target column: {a}")
        return a

pat = re.compile(rf"\bbf\s*[_-]?\s*{k}\b", flags=re.I)
for col in df.columns:
    if pat.search(str(col)):
        print(f" Found target column (regex match): {col}")
        return col

print(f" {target_bf} column not found automatically.")
print(" Available columns:", df.columns.tolist())
user_col = input(f" Enter exact column name for {target_bf}: ").strip()
return user_col

def preprocess_bf_target(y_series: pd.Series, target_label: str):
    y_numeric = pd.to_numeric(y_series, errors="coerce")
    valid_mask = y_numeric.notna() & np.isfinite(y_numeric)
    y_clean = y_numeric[valid_mask]

    if len(y_clean) == 0:
        raise ValueError(f" No valid values for {target_label}")

    print(f"\n TARGET ANALYSIS: {target_label}")
    print("-" * 40)
    print("Value distribution:\n", y_clean.value_counts().sort_index())

    y_binary = y_clean.astype(int)
    class_balance = y_binary.value_counts(normalize=True).round(3)
    print(f" Final class balance: {class_balance.to_dict()}")

    return y_binary, valid_mask

# =====
# 3) FEATURE ENGINEERING
# =====
def build_feature_matrix(df_clean: pd.DataFrame, target_col: str):
    print("\n FEATURE ENGINEERING")
    print("-" * 40)

    X = df_clean.drop(columns=[target_col], errors="ignore")
    numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()

    imputer = SimpleImputer(strategy="median")

```

```

X_imputed = imputer.fit_transform(X[numeric_cols])
X_imputed = pd.DataFrame(X_imputed, columns=numeric_cols, index=X.index)

non_constant_cols = [c for c in X_imputed.columns if X_imputed[c].nunique()
↳ 1]
X_final = X_imputed[non_constant_cols]

print(f" Final feature matrix: {X_final.shape} (rows, cols)")
return X_final, non_constant_cols

# =====
# 4) MODEL DEFINITIONS
# =====
def get_model_pipeline(model_type, n_samples=None):
    """Returns a pipeline with Scaler + Classifier based on model_type."""

    # Preprocessing (Scaling is MANDATORY for SVM/KNN)
    preprocessor = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])

    if model_type == "knn":
        # Dynamic K: Sqrt(N) is a good rule of thumb, but keep it odd
        k = int(np.sqrt(n_samples)) if n_samples else 5
        if k % 2 == 0: k += 1
        print(f" Configured KNN with n_neighbors={k}")

        clf = KNeighborsClassifier(n_neighbors=k, weights='distance', n_jobs=-1)

    elif model_type == "svm":
        print(" Configured SVM (RBF Kernel, C=1.0, Probability=True)")
        clf = SVC(
            kernel='rbf',
            C=1.0,
            class_weight='balanced',
            probability=True, # Needed for ROC Curves
            random_state=RANDOM_STATE
        )
    else:
        raise ValueError("model_type must be 'knn' or 'svm'")

    return Pipeline([("pre", preprocessor), ("clf", clf)])

# =====
# 5) CROSS-VALIDATION
# =====

```

```

def perform_cross_validation(X, y, model_type, cv_folds=10):
    print(f"\n PERFORMING {cv_folds}-FOLD CROSS-VALIDATION ({model_type}.
    ↪upper()))")
    print("=" * 50)

    pipe = get_model_pipeline(model_type, n_samples=len(X))

    scoring = {
        "accuracy": "accuracy", "precision": "precision", "recall": "recall",
        "f1": "f1", "roc_auc": "roc_auc", "average_precision": ↪
    ↪"average_precision"
    }

    cv_results = cross_validate(
        pipe, X, y,
        cv=StratifiedKFold(n_splits=cv_folds, shuffle=True, ↪
    ↪random_state=RANDOM_STATE),
        scoring=scoring,
        return_train_score=True,
        n_jobs=-1
    )

    print(" CROSS-VALIDATION RESULTS (Mean ± Std):")
    print("-" * 40)

    metrics_summary = {}
    for metric in scoring.keys():
        test_scores = cv_results[f"test_{metric}"]
        train_scores = cv_results[f"train_{metric}"]

        metrics_summary[metric] = {
            "test_mean": float(np.mean(test_scores)),
            "test_std": float(np.std(test_scores)),
            "train_mean": float(np.mean(train_scores))
        }

        print(f"{metric.upper():<15}: {np.mean(test_scores):.4f} ± {np.
    ↪std(test_scores):.4f}")

    return metrics_summary, cv_results

# =====
# 6) TRAINING
# =====
def create_validation_set(X, y, validation_size=0.2):
    print(f"\n CREATING SPLITS (Val={validation_size})")
    X_train_test, X_val, y_train_test, y_val = train_test_split(
        X, y, test_size=validation_size, stratify=y, random_state=RANDOM_STATE
    )

```

```

    )
    X_train, X_test, y_train, y_test = train_test_split(
        X_train_test, y_train_test, test_size=0.25, stratify=y_train_test,
        random_state=RANDOM_STATE
    )
    return X_train, X_test, X_val, y_train, y_test, y_val

def train_model_comprehensive(X, y, model_type, use_validation_set=True):
    print(f"\n MODEL TRAINING: {model_type.upper()}")
    print("-" * 40)

    if use_validation_set:
        X_train, X_test, X_val, y_train, y_test, y_val =
        create_validation_set(X, y)
    else:
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.
        25, stratify=y, random_state=RANDOM_STATE)
        X_val, y_val = None, None

    # Get pipeline with correct scaling + classifier
    pipe = get_model_pipeline(model_type, n_samples=len(X_train))

    print(" Fitting model...")
    pipe.fit(X_train, y_train)

    return pipe, X_train, X_test, X_val, y_train, y_test, y_val

# =====
# 7) EVALUATION & VISUALIZATION
# =====
def evaluate_model_metrics(model, X_test, y_test, dataset_name="Test"):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    metrics = {
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred, zero_division=0),
        "recall": recall_score(y_test, y_pred, zero_division=0),
        "f1": f1_score(y_test, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_test, y_proba),
        "pr_auc": average_precision_score(y_test, y_proba)
    }

    print("\n" + "=" * 60)
    print(f" {dataset_name.upper()} SET EVALUATION")
    print("=" * 60)
    for k, v in metrics.items():

```

```

        print(f" {k.capitalize():<10}: {v:.4f}")

    print(f"\n {dataset_name} Classification Report:\n",
    ↪classification_report(y_test, y_pred, digits=3))
    print(f" {dataset_name} Confusion Matrix:\n", confusion_matrix(y_test,
    ↪y_pred))

    return y_pred, y_proba, metrics

def create_visualizations(y_test, y_proba, model_type, output_dir, prefix,
    ↪cv_metrics=None):
    os.makedirs(output_dir, exist_ok=True)

    # 1. ROC & PR Curves
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

    fpr, tpr, _ = roc_curve(y_test, y_proba)
    roc_auc = roc_auc_score(y_test, y_proba)
    ax1.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}", linewidth=2, color="C0")
    ax1.plot([0, 1], [0, 1], "k--", alpha=0.5)
    ax1.set_title(f"ROC Curve ({model_type.upper()})")
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    prec, rec, _ = precision_recall_curve(y_test, y_proba)
    pr_auc = average_precision_score(y_test, y_proba)
    ax2.plot(rec, prec, label=f"AP = {pr_auc:.3f}", linewidth=2, color="C1")
    ax2.set_title(f"Precision-Recall Curve ({model_type.upper()})")
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(f"{output_dir}/{prefix}_roc_pr_curves.png", dpi=200)
    plt.show()

    # 2. Confusion Matrix
    y_pred = (y_proba >= 0.5).astype(int)
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Pred 0",
    ↪"Pred 1"], yticklabels=["True 0", "True 1"])
    plt.title(f"Confusion Matrix ({model_type.upper()})")
    plt.savefig(f"{output_dir}/{prefix}_confusion_matrix.png", dpi=200)
    plt.show()

    # 3. CV Comparison
    if cv_metrics:

```



```

names = list(cv_metrics.keys())
means = [cv_metrics[m]["test_mean"] for m in names]
stds = [cv_metrics[m]["test_std"] for m in names]

plt.figure(figsize=(10, 5))
bars = plt.bar(names, means, yerr=stds, capsize=5, alpha=0.7)
for i, b in enumerate(bars): b.set_color(f"C{i}")
plt.title(f"Cross-Validation Performance ({model_type.upper()})")
plt.ylim(0, 1)
plt.grid(axis='y', alpha=0.3)
plt.savefig(f"{output_dir}/{prefix}_cv_results.png", dpi=200)
plt.show()

# =====
# 8) SAVING RESULTS
# =====
def save_results(model, metrics, df_clean, target_col, feature_names,
    ↪output_dir, prefix, cv_metrics=None):
    print(f"\n SAVING OUTPUTS to {output_dir}/ ...")

    # 1. Metrics Text File
    with open(f"{output_dir}/{prefix}_metrics.txt", "w") as f:
        f.write(f"MODEL RESULTS: {prefix.upper()}\n{'='*40}\n")
        f.write("TEST SET METRICS:\n")
        for k, v in metrics.items(): f.write(f"{k}: {v:.4f}\n")

        if cv_metrics:
            f.write("\nCROSS-VALIDATION:\n")
            for k, v in cv_metrics.items():
                f.write(f"{k}: {v['test_mean']:.4f} ± {v['test_std']:.4f}\n")

    # 2. Excel Predictions
    X_full = df_clean.drop(columns=[target_col], errors="ignore").
    ↪reindex(columns=feature_names)
    preds = model.predict_proba(X_full)[:, 1]

    out_df = df_clean.copy()
    out_df[f"{prefix}_Prob"] = preds
    out_df[f"{prefix}_Pred"] = (preds >= 0.5).astype(int)
    out_df.to_excel(f"{output_dir}/{prefix}_predictions.xlsx", index=False)

    # 3. Model
    joblib.dump(model, f"{output_dir}/{prefix}_model.joblib")
    print(" All files saved successfully.")

# =====
# 9) MAIN PIPELINE

```

```

# =====
def run_comprehensive_model_analysis(target_bf="BF1", model_type="svm",
    ↪df=None):
    """
    Run pipeline for SVM or KNN.
    model_type: 'svm' or 'knn'
    """
    try:
        model_type = model_type.lower()
        prefix = f"{target_bf}_{model_type}"
        output_dir = f"{prefix}_outputs"

        print(f" STARTING ANALYSIS: {model_type.upper()} on {target_bf}")

        if df is None:
            try: df = load_excel_from_colab()
            except: df = load_excel_locally()

        # Target Setup
        target_col = detect_bf_target(df, target_bf)
        y, valid_mask = preprocess_bf_target(df[target_col], target_col)
        df_clean = df.loc[valid_mask].copy()

        # Features
        X_final, feature_names = build_feature_matrix(df_clean, target_col)

        # 1. CV
        cv_metrics, _ = perform_cross_validation(X_final, y, model_type,
    ↪cv_folds=10)

        # 2. Train
        model, X_train, X_test, X_val, y_train, y_test, y_val =
    ↪train_model_comprehensive(
            X_final, y, model_type, use_validation_set=True
        )

        # 3. Eval
        _, y_proba, test_metrics = evaluate_model_metrics(model, X_test,
    ↪y_test, "Test")

        # 4. Viz
        create_visualizations(y_test, y_proba, model_type, output_dir, prefix,
    ↪cv_metrics)

        # 5. Save
        save_results(model, test_metrics, df_clean, target_col, feature_names,
    ↪output_dir, prefix, cv_metrics)

```

```

        print("\n DONE.")
        return model

    except Exception as e:
        print(f"\n Error: {e}")
        return None

# ===== USAGE =====
if __name__ == "__main__":
    # Example 1: Run SVM on BF6 (Default)
    run_comprehensive_model_analysis(target_bf="BF6", model_type="svm")

    # Example 2: Run KNN (Uncomment to use)
    # run_comprehensive_model_analysis(target_bf="BF6", model_type="knn")

```

STARTING ANALYSIS: SVM on BF6

Please select the Excel file with your encoded dataset...

<IPython.core.display.HTML object>

Saving rf_dataset_encoded_dataset.xlsx to rf_dataset_encoded_dataset (3).xlsx

Loaded dataset: (1002, 85) (rows, cols)

BF6 column not found automatically.

Available columns: ['D', 'AY', 'AZ', 'BA', 'BB', 'BC', 'BD', 'BE_01', 'BE_02', 'BE_03', 'BE_04', 'BE_05', 'BE_06', 'BE_07', 'BE_08', 'BE_09', 'BE_10', 'BF', 'BG_01', 'BG_02', 'BG_03', 'BG_04', 'BG_05', 'BG_06', 'BH', 'BI', 'BN', 'BO', 'BP', 'BQ', 'BR', 'BS', 'BT', 'BU', 'BV', 'BW_01', 'BW_02', 'BW_03', 'BW_04', 'BW_05', 'BX', 'BY', 'CB', 'CD', 'CE', 'CF', 'CG', 'CH', 'CI', 'CK', 'CL', 'CM_01', 'CM_02', 'CM_03', 'CM_04', 'CM_05', 'CM_06', 'CN', 'CO_01', 'CO_02', 'CO_03', 'CO_04', 'CO_05', 'CO_06', 'CO_07', 'CO_08', 'CO_09', 'CP', 'CR_01', 'CR_02', 'CR_03', 'CR_04', 'CS', 'CT_01', 'CT_02', 'CT_03', 'CT_04', 'CT_05', 'CT_06', 'CU', 'Occupation', 'Financial Behavior', 'Family', 'Lifestyle', 'Digital Literacy']

Enter exact column name for BF6: BG_04

TARGET ANALYSIS: BG_04

Value distribution:

BG_04

0 605

1 397

Name: count, dtype: int64

Final class balance: {0: 0.604, 1: 0.396}

FEATURE ENGINEERING

Final feature matrix: (1002, 84) (rows, cols)

PERFORMING 10-FOLD CROSS-VALIDATION (SVM)

Configured SVM (RBF Kernel, C=1.0, Probability=True)
CROSS-VALIDATION RESULTS (Mean $\pm$ Std):

ACCURACY : 0.8344 $\pm$ 0.0277
PRECISION : 0.7500 $\pm$ 0.0385
RECALL : 0.8764 $\pm$ 0.0432
F1 : 0.8075 $\pm$ 0.0315
ROC_AUC : 0.9239 $\pm$ 0.0326
AVERAGE_PRECISION: 0.8827 $\pm$ 0.0525

MODEL TRAINING: SVM

CREATING SPLITS (Val=0.2)

Configured SVM (RBF Kernel, C=1.0, Probability=True)
Fitting model...

TEST SET EVALUATION

=====

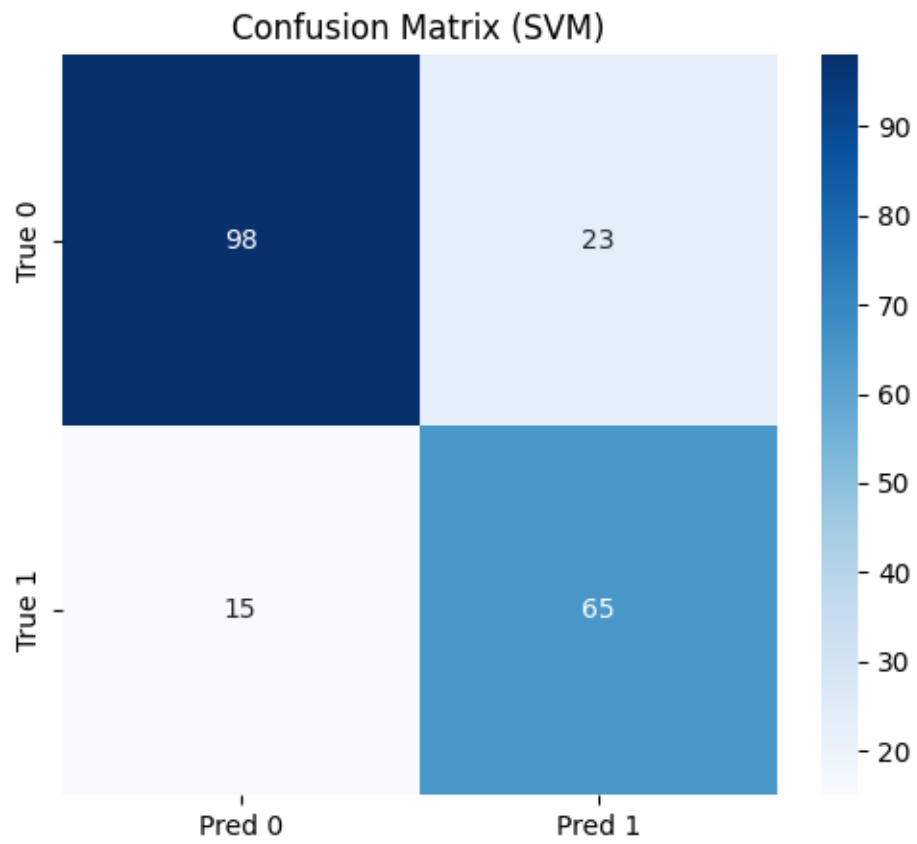
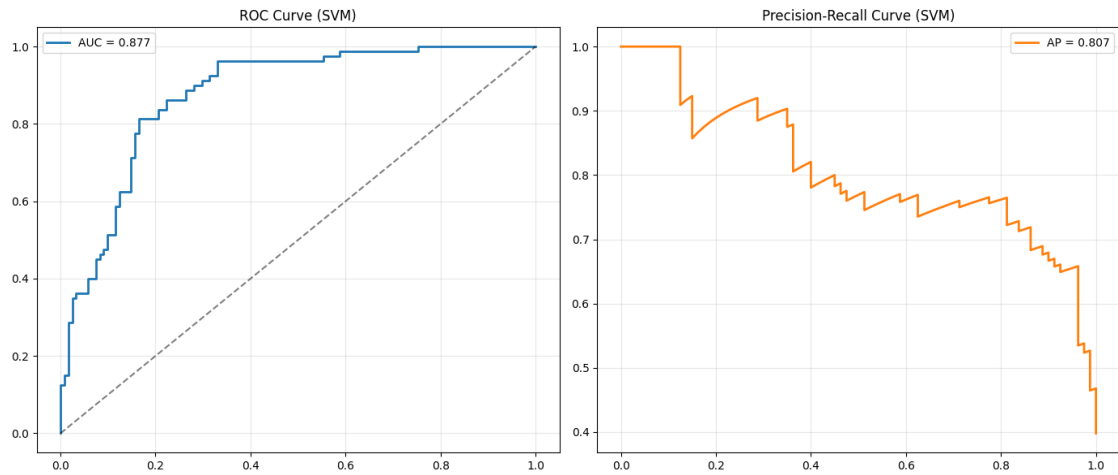
Accuracy : 0.8010
Precision : 0.7128
Recall : 0.8375
F1 : 0.7701
Roc_auc : 0.8769
Pr_auc : 0.8066

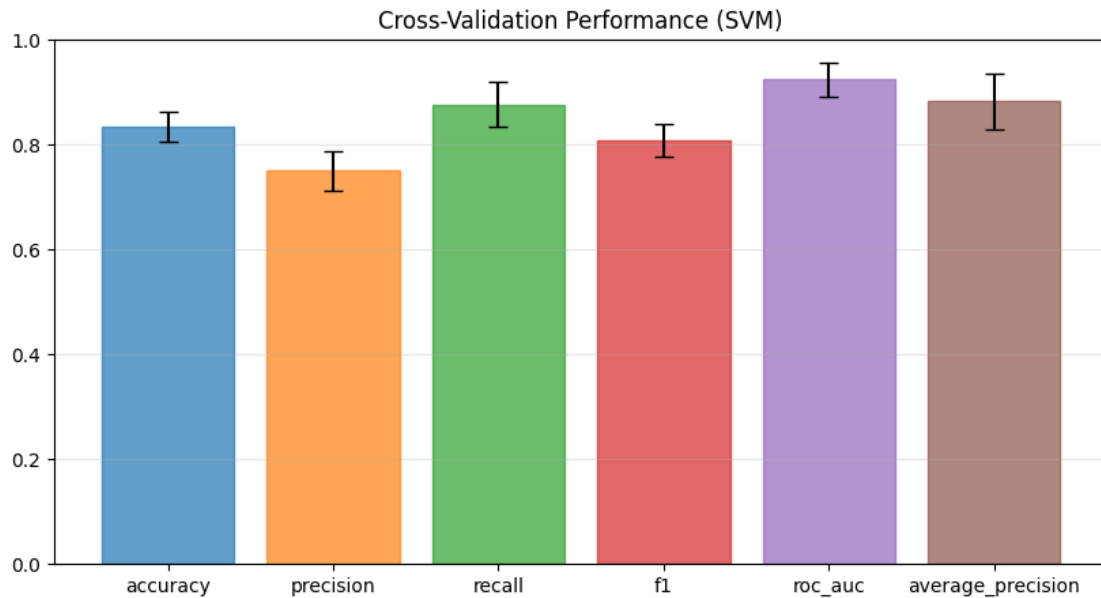
Test Classification Report:

	precision	recall	f1-score	support
0	0.879	0.777	0.825	121
1	0.713	0.838	0.770	80
accuracy			0.801	201
macro avg	0.796	0.807	0.797	201
weighted avg	0.813	0.801	0.803	201

Test Confusion Matrix:

[[94 27]
[13 67]]





SAVING OUTPUTS to BF6_svm_outputs/ ...
All files saved successfully.

DONE.

```
[7]: """
Reusable KNN & SVM Pipeline for Microfinance Impact Analysis
-----
Matches the output style of the Random Forest script (Graphs, Validations,
↳Excel).

Usage:
    run_comprehensive_model_analysis(target_bf="BF1", model_type="svm")
    run_comprehensive_model_analysis(target_bf="BF3", model_type="knn")
"""

# =====
# 0) IMPORTS
# =====
import os, io, re, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```

from sklearn.model_selection import train_test_split, StratifiedKFold,
↳ cross_validate
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score, roc_curve,
    precision_recall_curve, average_precision_score,
    accuracy_score, precision_score, recall_score, f1_score
)
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import joblib

RANDOM_STATE = 42

# =====
# 1) DATA LOADING
# =====
def load_excel_from_colab():
    """Upload Excel in Google Colab."""
    try:
        from google.colab import files
        print(" Please select the Excel file with your encoded dataset...")
        uploaded = files.upload()
        path = list(uploaded.keys())[0]
        df = pd.read_excel(path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        raise RuntimeError("Colab upload failed.") from e

def load_excel_locally():
    """Upload Excel when running locally."""
    try:
        import tkinter as tk
        from tkinter import filedialog
        print(" Please select the Excel file...")
        root = tk.Tk()
        root.withdraw()
        file_path = filedialog.askopenfilename(
            title="Select Excel file",
            filetypes=[("Excel files", "*.xlsx *.xls")]
        )
        if not file_path:
            raise ValueError("No file selected")
    
```

```

    df = pd.read_excel(file_path)
    print(f" Loaded dataset: {df.shape} (rows, cols)")
    return df
except Exception as e:
    print(f" File dialog failed: {e}")
    file_path = input(" Enter full path to your Excel file: ")
    df = pd.read_excel(file_path)
    print(f" Loaded dataset: {df.shape} (rows, cols)")
    return df

# =====
# 2) TARGET SELECTION
# =====
def detect_bf_target(df: pd.DataFrame, target_bf: str = "BF1") -> str:
    m = re.match(r"^s*BF\s*(\d+)\s*$", str(target_bf), flags=re.I)
    if not m:
        raise ValueError(f"target_bf must look like 'BF1', 'BF3', etc. Got: {target_bf}")

    k = m.group(1)
    aliases = [f"BF{k}", f"bf{k}", f"BF_{k}", f"bf_{k}"]

    for a in aliases:
        if a in df.columns:
            print(f" Found target column: {a}")
            return a

    pat = re.compile(rf"\bbf\s*[_-]?s*{k}\b", flags=re.I)
    for col in df.columns:
        if pat.search(str(col)):
            print(f" Found target column (regex match): {col}")
            return col

    print(f" {target_bf} column not found automatically.")
    print(" Available columns:", df.columns.tolist())
    user_col = input(f" Enter exact column name for {target_bf}: ").strip()
    return user_col

def preprocess_bf_target(y_series: pd.Series, target_label: str):
    y_numeric = pd.to_numeric(y_series, errors="coerce")
    valid_mask = y_numeric.notna() & np.isfinite(y_numeric)
    y_clean = y_numeric[valid_mask]

    if len(y_clean) == 0:
        raise ValueError(f" No valid values for {target_label}")

    print(f"\n TARGET ANALYSIS: {target_label}")

```



```

print("-" * 40)
print("Value distribution:\n", y_clean.value_counts().sort_index())

y_binary = y_clean.astype(int)
class_balance = y_binary.value_counts(normalize=True).round(3)
print(f" Final class balance: {class_balance.to_dict()}")

return y_binary, valid_mask

# =====
# 3) FEATURE ENGINEERING
# =====
def build_feature_matrix(df_clean: pd.DataFrame, target_col: str):
    print("\n FEATURE ENGINEERING")
    print("-" * 40)

    X = df_clean.drop(columns=[target_col], errors="ignore")
    numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()

    imputer = SimpleImputer(strategy="median")
    X_imputed = imputer.fit_transform(X[numeric_cols])
    X_imputed = pd.DataFrame(X_imputed, columns=numeric_cols, index=X.index)

    non_constant_cols = [c for c in X_imputed.columns if X_imputed[c].nunique()
↪ > 1]
    X_final = X_imputed[non_constant_cols]

    print(f" Final feature matrix: {X_final.shape} (rows, cols)")
    return X_final, non_constant_cols

# =====
# 4) MODEL DEFINITIONS
# =====
def get_model_pipeline(model_type, n_samples=None):
    """Returns a pipeline with Scaler + Classifier based on model_type."""

    # Preprocessing (Scaling is MANDATORY for SVM/KNN)
    preprocessor = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])

    if model_type == "knn":
        # Dynamic K: Sqrt(N) is a good rule of thumb, but keep it odd
        k = int(np.sqrt(n_samples)) if n_samples else 5
        if k % 2 == 0: k += 1
        print(f" Configured KNN with n_neighbors={k}")

```

```

        clf = KNeighborsClassifier(n_neighbors=k, weights='distance', n_jobs=-1)

    elif model_type == "svm":
        print(" Configured SVM (RBF Kernel, C=1.0, Probability=True)")
        clf = SVC(
            kernel='rbf',
            C=1.0,
            class_weight='balanced',
            probability=True, # Needed for ROC Curves
            random_state=RANDOM_STATE
        )
    else:
        raise ValueError("model_type must be 'knn' or 'svm'")

    return Pipeline([("pre", preprocessor), ("clf", clf)])

# =====
# 5) CROSS-VALIDATION
# =====
def perform_cross_validation(X, y, model_type, cv_folds=10):
    print(f"\n PERFORMING {cv_folds}-FOLD CROSS-VALIDATION ({model_type}.
    ↪upper()))")
    print("=" * 50)

    pipe = get_model_pipeline(model_type, n_samples=len(X))

    scoring = {
        "accuracy": "accuracy", "precision": "precision", "recall": "recall",
        "f1": "f1", "roc_auc": "roc_auc", "average_precision": ↪
    ↪"average_precision"
    }

    cv_results = cross_validate(
        pipe, X, y,
        cv=StratifiedKFold(n_splits=cv_folds, shuffle=True, ↪
    ↪random_state=RANDOM_STATE),
        scoring=scoring,
        return_train_score=True,
        n_jobs=-1
    )

    print(" CROSS-VALIDATION RESULTS (Mean ± Std):")
    print("-" * 40)

    metrics_summary = {}
    for metric in scoring.keys():

```

```

test_scores = cv_results[f"test_{metric}"]
train_scores = cv_results[f"train_{metric}"]

metrics_summary[metric] = {
    "test_mean": float(np.mean(test_scores)),
    "test_std": float(np.std(test_scores)),
    "train_mean": float(np.mean(train_scores))
}
print(f"{metric.upper():<15}: {np.mean(test_scores):.4f} ± {np.
↪std(test_scores):.4f}")

return metrics_summary, cv_results

# =====
# 6) TRAINING
# =====
def create_validation_set(X, y, validation_size=0.2):
    print(f"\n CREATING SPLITS (Val={validation_size})")
    X_train_test, X_val, y_train_test, y_val = train_test_split(
        X, y, test_size=validation_size, stratify=y, random_state=RANDOM_STATE
    )
    X_train, X_test, y_train, y_test = train_test_split(
        X_train_test, y_train_test, test_size=0.25, stratify=y_train_test, ↪
↪random_state=RANDOM_STATE
    )
    return X_train, X_test, X_val, y_train, y_test, y_val

def train_model_comprehensive(X, y, model_type, use_validation_set=True):
    print(f"\n MODEL TRAINING: {model_type.upper()}")
    print("-" * 40)

    if use_validation_set:
        X_train, X_test, X_val, y_train, y_test, y_val = ↪
↪create_validation_set(X, y)
    else:
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.
↪25, stratify=y, random_state=RANDOM_STATE)
        X_val, y_val = None, None

    # Get pipeline with correct scaling + classifier
    pipe = get_model_pipeline(model_type, n_samples=len(X_train))

    print(" Fitting model...")
    pipe.fit(X_train, y_train)

    return pipe, X_train, X_test, X_val, y_train, y_test, y_val

```

```

# =====
# 7) EVALUATION & VISUALIZATION
# =====
def evaluate_model_metrics(model, X_test, y_test, dataset_name="Test"):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    metrics = {
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred, zero_division=0),
        "recall": recall_score(y_test, y_pred, zero_division=0),
        "f1": f1_score(y_test, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_test, y_proba),
        "pr_auc": average_precision_score(y_test, y_proba)
    }

    print("\n" + "=" * 60)
    print(f" {dataset_name.upper()} SET EVALUATION")
    print("=" * 60)
    for k, v in metrics.items():
        print(f" {k.capitalize():<10}: {v:.4f}")

    print(f"\n {dataset_name} Classification Report:\n",
    ↪classification_report(y_test, y_pred, digits=3))
    print(f" {dataset_name} Confusion Matrix:\n", confusion_matrix(y_test,
    ↪y_pred))

    return y_pred, y_proba, metrics

def create_visualizations(y_test, y_proba, model_type, output_dir, prefix,
    ↪cv_metrics=None):
    os.makedirs(output_dir, exist_ok=True)

    # 1. ROC & PR Curves
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

    fpr, tpr, _ = roc_curve(y_test, y_proba)
    roc_auc = roc_auc_score(y_test, y_proba)
    ax1.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}", linewidth=2, color="C0")
    ax1.plot([0, 1], [0, 1], "k--", alpha=0.5)
    ax1.set_title(f"ROC Curve ({model_type.upper()})")
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    prec, rec, _ = precision_recall_curve(y_test, y_proba)
    pr_auc = average_precision_score(y_test, y_proba)
    ax2.plot(rec, prec, label=f"AP = {pr_auc:.3f}", linewidth=2, color="C1")

```

```

ax2.set_title(f"Precision-Recall Curve ({model_type.upper()})")
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_roc_pr_curves.png", dpi=200)
plt.show()

# 2. Confusion Matrix
y_pred = (y_proba >= 0.5).astype(int)
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Pred 0", "Pred 1"], yticklabels=["True 0", "True 1"])
plt.title(f"Confusion Matrix ({model_type.upper()})")
plt.savefig(f"{output_dir}/{prefix}_confusion_matrix.png", dpi=200)
plt.show()

# 3. CV Comparison
if cv_metrics:
    names = list(cv_metrics.keys())
    means = [cv_metrics[m]["test_mean"] for m in names]
    stds = [cv_metrics[m]["test_std"] for m in names]

    plt.figure(figsize=(10, 5))
    bars = plt.bar(names, means, yerr=stds, capsize=5, alpha=0.7)
    for i, b in enumerate(bars): b.set_color(f"C{i}")
    plt.title(f"Cross-Validation Performance ({model_type.upper()})")
    plt.ylim(0, 1)
    plt.grid(axis='y', alpha=0.3)
    plt.savefig(f"{output_dir}/{prefix}_cv_results.png", dpi=200)
    plt.show()

# =====
# 8) SAVING RESULTS
# =====
def save_results(model, metrics, df_clean, target_col, feature_names,
                 output_dir, prefix, cv_metrics=None):
    print(f"\n SAVING OUTPUTS to {output_dir}/ ...")

    # 1. Metrics Text File
    with open(f"{output_dir}/{prefix}_metrics.txt", "w") as f:
        f.write(f"MODEL RESULTS: {prefix.upper()}\n{'='*40}\n")
        f.write("TEST SET METRICS:\n")
        for k, v in metrics.items(): f.write(f"{k}: {v:.4f}\n")

    if cv_metrics:

```

```

        f.write("\nCROSS-VALIDATION:\n")
        for k, v in cv_metrics.items():
            f.write(f"{k}: {v['test_mean']:.4f} ± {v['test_std']:.4f}\n")

    # 2. Excel Predictions
    X_full = df_clean.drop(columns=[target_col], errors="ignore").
    ↪reindex(columns=feature_names)
    preds = model.predict_proba(X_full)[: , 1]

    out_df = df_clean.copy()
    out_df[f"{prefix}_Prob"] = preds
    out_df[f"{prefix}_Pred"] = (preds >= 0.5).astype(int)
    out_df.to_excel(f"{output_dir}/{prefix}_predictions.xlsx", index=False)

    # 3. Model
    joblib.dump(model, f"{output_dir}/{prefix}_model.joblib")
    print(" All files saved successfully.")

# =====
# 9) MAIN PIPELINE
# =====
def run_comprehensive_model_analysis(target_bf="BF1", model_type="svm",
    ↪df=None):
    """
    Run pipeline for SVM or KNN.
    model_type: 'svm' or 'knn'
    """
    try:
        model_type = model_type.lower()
        prefix = f"{target_bf}_{model_type}"
        output_dir = f"{prefix}_outputs"

        print(f" STARTING ANALYSIS: {model_type.upper()} on {target_bf}")

        if df is None:
            try: df = load_excel_from_colab()
            except: df = load_excel_locally()

        # Target Setup
        target_col = detect_bf_target(df, target_bf)
        y, valid_mask = preprocess_bf_target(df[target_col], target_col)
        df_clean = df.loc[valid_mask].copy()

        # Features
        X_final, feature_names = build_feature_matrix(df_clean, target_col)

    # 1. CV

```

```

        cv_metrics, _ = perform_cross_validation(X_final, y, model_type,
↪cv_folds=10)

        # 2. Train
        model, X_train, X_test, X_val, y_train, y_test, y_val =
↪train_model_comprehensive(
            X_final, y, model_type, use_validation_set=True
        )

        # 3. Eval
        _, y_proba, test_metrics = evaluate_model_metrics(model, X_test,
↪y_test, "Test")

        # 4. Viz
        create_visualizations(y_test, y_proba, model_type, output_dir, prefix,
↪cv_metrics)

        # 5. Save
        save_results(model, test_metrics, df_clean, target_col, feature_names,
↪output_dir, prefix, cv_metrics)

        print("\n DONE.")
        return model

    except Exception as e:
        print(f"\n Error: {e}")
        return None

# ===== USAGE =====
if __name__ == "__main__":
    # Example 1: Run SVM on BF6 (Default)
    run_comprehensive_model_analysis(target_bf="BF6", model_type="svm")

    # Example 2: Run KNN (Uncomment to use)
    # run_comprehensive_model_analysis(target_bf="BF6", model_type="knn")

```

STARTING ANALYSIS: SVM on BF6

Please select the Excel file with your encoded dataset...

<IPython.core.display.HTML object>

Saving rf_dataset_encoded_dataset.xlsx to rf_dataset_encoded_dataset (4).xlsx

Loaded dataset: (1002, 85) (rows, cols)

BF6 column not found automatically.

Available columns: ['D', 'AY', 'AZ', 'BA', 'BB', 'BC', 'BD', 'BE_01', 'BE_02', 'BE_03', 'BE_04', 'BE_05', 'BE_06', 'BE_07', 'BE_08', 'BE_09', 'BE_10', 'BF', 'BG_01', 'BG_02', 'BG_03', 'BG_04', 'BG_05', 'BG_06', 'BH', 'BI', 'BN', 'BO', 'BP', 'BQ', 'BR', 'BS', 'BT', 'BU', 'BV', 'BW_01', 'BW_02', 'BW_03', 'BW_04',

```
'BW_05', 'BX', 'BY', 'CB', 'CD', 'CE', 'CF', 'CG', 'CH', 'CI', 'CK', 'CL',  
'CM_01', 'CM_02', 'CM_03', 'CM_04', 'CM_05', 'CM_06', 'CN', 'CO_01', 'CO_02',  
'CO_03', 'CO_04', 'CO_05', 'CO_06', 'CO_07', 'CO_08', 'CO_09', 'CP', 'CR_01',  
'CR_02', 'CR_03', 'CR_04', 'CS', 'CT_01', 'CT_02', 'CT_03', 'CT_04', 'CT_05',  
'CT_06', 'CU', 'Occupation', 'Financial Behavior', 'Family', 'Lifestyle',  
'Digital Literacy']
```

Enter exact column name for BF6: BG_05

TARGET ANALYSIS: BG_05

Value distribution:

BG_05

0 657

1 345

Name: count, dtype: int64

Final class balance: {0: 0.656, 1: 0.344}

FEATURE ENGINEERING

Final feature matrix: (1002, 84) (rows, cols)

PERFORMING 10-FOLD CROSS-VALIDATION (SVM)

=====

Configured SVM (RBF Kernel, C=1.0, Probability=True)

CROSS-VALIDATION RESULTS (Mean ± Std):

ACCURACY : 0.7934 ± 0.0462
PRECISION : 0.6838 ± 0.0701
RECALL : 0.7508 ± 0.0738
F1 : 0.7144 ± 0.0641
ROC_AUC : 0.8870 ± 0.0335
AVERAGE_PRECISION: 0.8192 ± 0.0669

MODEL TRAINING: SVM

CREATING SPLITS (Val=0.2)

Configured SVM (RBF Kernel, C=1.0, Probability=True)

Fitting model...

=====

TEST SET EVALUATION

=====

Accuracy : 0.8209
Precision : 0.7324
Recall : 0.7536
F1 : 0.7429
Roc_auc : 0.8622

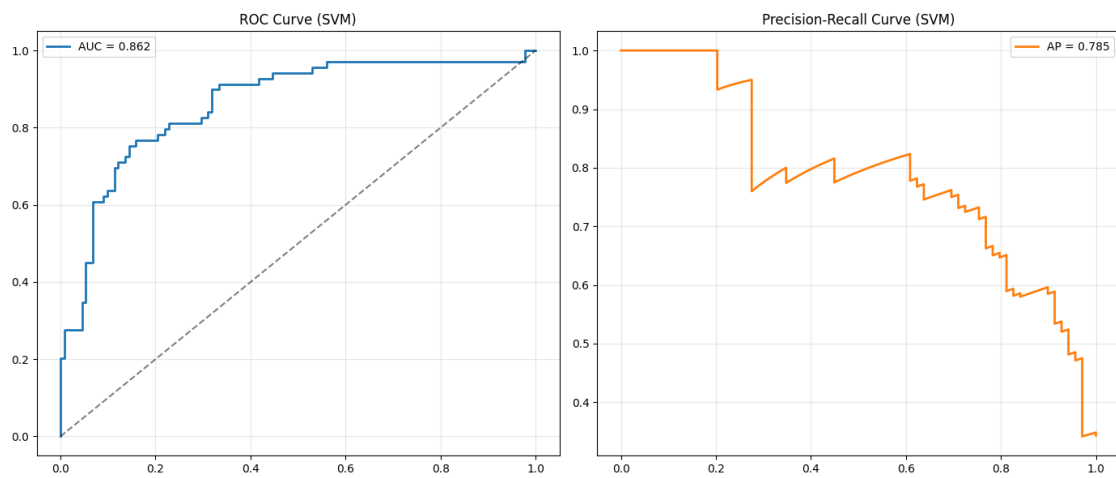
Pr_auc : 0.7849

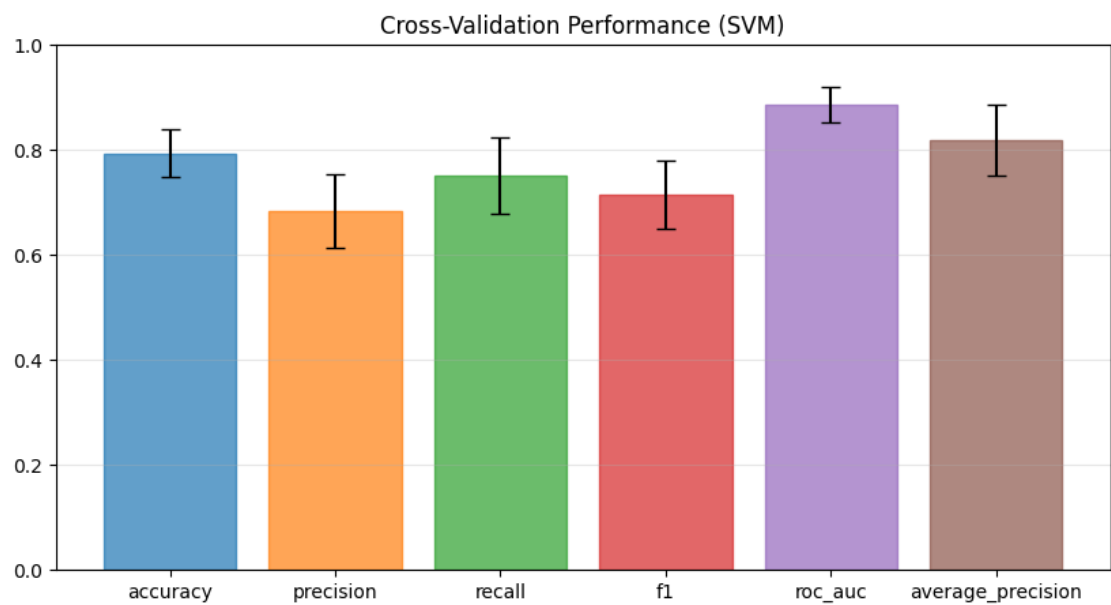
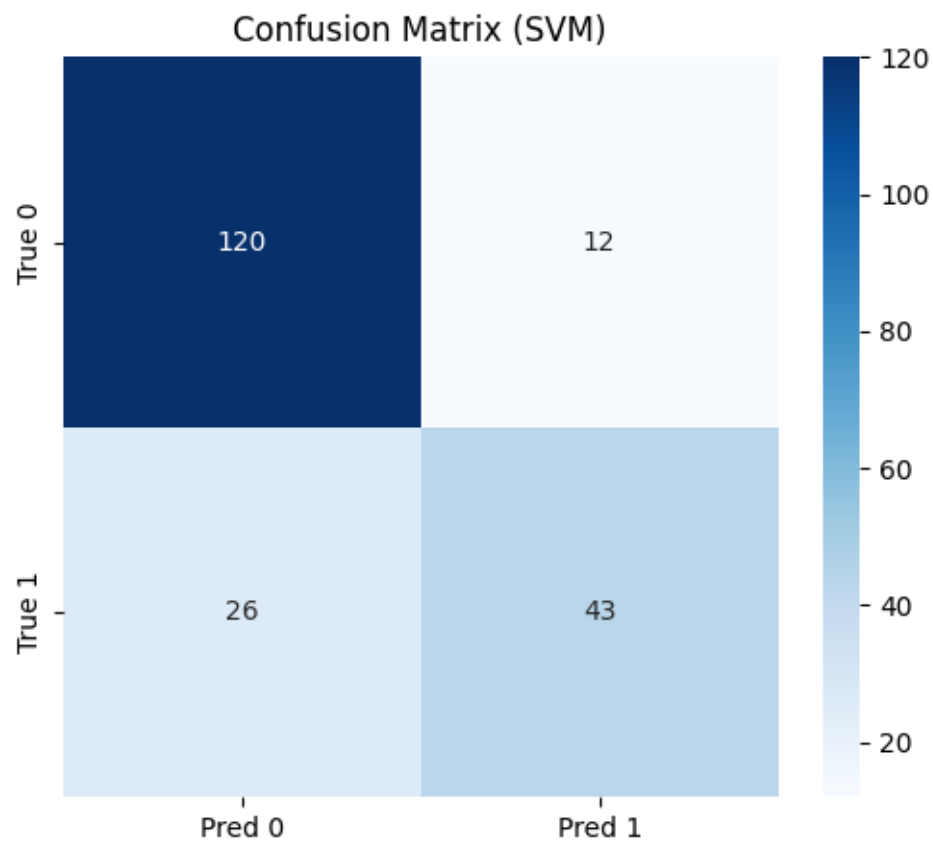
Test Classification Report:

	precision	recall	f1-score	support
0	0.869	0.856	0.863	132
1	0.732	0.754	0.743	69
accuracy			0.821	201
macro avg	0.801	0.805	0.803	201
weighted avg	0.822	0.821	0.821	201

Test Confusion Matrix:

```
[[113  19]
 [ 17  52]]
```





SAVING OUTPUTS to BF6_svm_outputs/ ...
All files saved successfully.

DONE.

```
[8]: """
Reusable KNN & SVM Pipeline for Microfinance Impact Analysis
-----
Matches the output style of the Random Forest script (Graphs, Validations,
↳Excel).

Usage:
    run_comprehensive_model_analysis(target_bf="BF1", model_type="svm")
    run_comprehensive_model_analysis(target_bf="BF3", model_type="knn")
"""

# =====
# 0) IMPORTS
# =====
import os, io, re, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold,
↳cross_validate
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score, roc_curve,
    precision_recall_curve, average_precision_score,
    accuracy_score, precision_score, recall_score, f1_score
)
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import joblib

RANDOM_STATE = 42

# =====
# 1) DATA LOADING
# =====
```

```

def load_excel_from_colab():
    """Upload Excel in Google Colab."""
    try:
        from google.colab import files
        print(" Please select the Excel file with your encoded dataset...")
        uploaded = files.upload()
        path = list(uploaded.keys())[0]
        df = pd.read_excel(path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        raise RuntimeError("Colab upload failed.") from e

def load_excel_locally():
    """Upload Excel when running locally."""
    try:
        import tkinter as tk
        from tkinter import filedialog
        print(" Please select the Excel file...")
        root = tk.Tk()
        root.withdraw()
        file_path = filedialog.askopenfilename(
            title="Select Excel file",
            filetypes=[("Excel files", "*.xlsx *.xls")]
        )
        if not file_path:
            raise ValueError("No file selected")
        df = pd.read_excel(file_path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        print(f" File dialog failed: {e}")
        file_path = input(" Enter full path to your Excel file: ")
        df = pd.read_excel(file_path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df

# =====
# 2) TARGET SELECTION
# =====
def detect_bf_target(df: pd.DataFrame, target_bf: str = "BF1") -> str:
    m = re.match(r"^\s*BF\s*(\d+)\s*$", str(target_bf), flags=re.I)
    if not m:
        raise ValueError(f"target_bf must look like 'BF1', 'BF3', etc. Got: {target_bf}")

    k = m.group(1)

```

```

aliases = ["BF{k}", f"bf{k}", f"BF_{k}", f"bf_{k}"]

for a in aliases:
    if a in df.columns:
        print(f" Found target column: {a}")
        return a

pat = re.compile(rf"\bbf\s*[_-]?\s*{k}\b", flags=re.I)
for col in df.columns:
    if pat.search(str(col)):
        print(f" Found target column (regex match): {col}")
        return col

print(f" {target_bf} column not found automatically.")
print(" Available columns:", df.columns.tolist())
user_col = input(f" Enter exact column name for {target_bf}: ").strip()
return user_col

def preprocess_bf_target(y_series: pd.Series, target_label: str):
    y_numeric = pd.to_numeric(y_series, errors="coerce")
    valid_mask = y_numeric.notna() & np.isfinite(y_numeric)
    y_clean = y_numeric[valid_mask]

    if len(y_clean) == 0:
        raise ValueError(f" No valid values for {target_label}")

    print(f"\n TARGET ANALYSIS: {target_label}")
    print("-" * 40)
    print("Value distribution:\n", y_clean.value_counts().sort_index())

    y_binary = y_clean.astype(int)
    class_balance = y_binary.value_counts(normalize=True).round(3)
    print(f" Final class balance: {class_balance.to_dict()}")

    return y_binary, valid_mask

# =====
# 3) FEATURE ENGINEERING
# =====
def build_feature_matrix(df_clean: pd.DataFrame, target_col: str):
    print("\n FEATURE ENGINEERING")
    print("-" * 40)

    X = df_clean.drop(columns=[target_col], errors="ignore")
    numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()

    imputer = SimpleImputer(strategy="median")

```

```

X_imputed = imputer.fit_transform(X[numeric_cols])
X_imputed = pd.DataFrame(X_imputed, columns=numeric_cols, index=X.index)

non_constant_cols = [c for c in X_imputed.columns if X_imputed[c].nunique()
↳ > 1]
X_final = X_imputed[non_constant_cols]

print(f" Final feature matrix: {X_final.shape} (rows, cols)")
return X_final, non_constant_cols

# =====
# 4) MODEL DEFINITIONS
# =====
def get_model_pipeline(model_type, n_samples=None):
    """Returns a pipeline with Scaler + Classifier based on model_type."""

    # Preprocessing (Scaling is MANDATORY for SVM/KNN)
    preprocessor = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])

    if model_type == "knn":
        # Dynamic K: Sqrt(N) is a good rule of thumb, but keep it odd
        k = int(np.sqrt(n_samples)) if n_samples else 5
        if k % 2 == 0: k += 1
        print(f" Configured KNN with n_neighbors={k}")

        clf = KNeighborsClassifier(n_neighbors=k, weights='distance', n_jobs=-1)

    elif model_type == "svm":
        print(" Configured SVM (RBF Kernel, C=1.0, Probability=True)")
        clf = SVC(
            kernel='rbf',
            C=1.0,
            class_weight='balanced',
            probability=True, # Needed for ROC Curves
            random_state=RANDOM_STATE
        )
    else:
        raise ValueError("model_type must be 'knn' or 'svm'")

    return Pipeline([("pre", preprocessor), ("clf", clf)])

# =====
# 5) CROSS-VALIDATION
# =====

```

```

def perform_cross_validation(X, y, model_type, cv_folds=10):
    print(f"\n PERFORMING {cv_folds}-FOLD CROSS-VALIDATION ({model_type}.
    ↪upper()))")
    print("=" * 50)

    pipe = get_model_pipeline(model_type, n_samples=len(X))

    scoring = {
        "accuracy": "accuracy", "precision": "precision", "recall": "recall",
        "f1": "f1", "roc_auc": "roc_auc", "average_precision": ↪
    ↪"average_precision"
    }

    cv_results = cross_validate(
        pipe, X, y,
        cv=StratifiedKFold(n_splits=cv_folds, shuffle=True, ↪
    ↪random_state=RANDOM_STATE),
        scoring=scoring,
        return_train_score=True,
        n_jobs=-1
    )

    print(" CROSS-VALIDATION RESULTS (Mean ± Std):")
    print("-" * 40)

    metrics_summary = {}
    for metric in scoring.keys():
        test_scores = cv_results[f"test_{metric}"]
        train_scores = cv_results[f"train_{metric}"]

        metrics_summary[metric] = {
            "test_mean": float(np.mean(test_scores)),
            "test_std": float(np.std(test_scores)),
            "train_mean": float(np.mean(train_scores))
        }

        print(f"{metric.upper():<15}: {np.mean(test_scores):.4f} ± {np.
    ↪std(test_scores):.4f}")

    return metrics_summary, cv_results

# =====
# 6) TRAINING
# =====
def create_validation_set(X, y, validation_size=0.2):
    print(f"\n CREATING SPLITS (Val={validation_size})")
    X_train_test, X_val, y_train_test, y_val = train_test_split(
        X, y, test_size=validation_size, stratify=y, random_state=RANDOM_STATE
    )

```

```

    )
    X_train, X_test, y_train, y_test = train_test_split(
        X_train_test, y_train_test, test_size=0.25, stratify=y_train_test,
        random_state=RANDOM_STATE
    )
    return X_train, X_test, X_val, y_train, y_test, y_val

def train_model_comprehensive(X, y, model_type, use_validation_set=True):
    print(f"\n MODEL TRAINING: {model_type.upper()}")
    print("-" * 40)

    if use_validation_set:
        X_train, X_test, X_val, y_train, y_test, y_val =
        create_validation_set(X, y)
    else:
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.
        25, stratify=y, random_state=RANDOM_STATE)
        X_val, y_val = None, None

    # Get pipeline with correct scaling + classifier
    pipe = get_model_pipeline(model_type, n_samples=len(X_train))

    print(" Fitting model...")
    pipe.fit(X_train, y_train)

    return pipe, X_train, X_test, X_val, y_train, y_test, y_val

# =====
# 7) EVALUATION & VISUALIZATION
# =====
def evaluate_model_metrics(model, X_test, y_test, dataset_name="Test"):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    metrics = {
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred, zero_division=0),
        "recall": recall_score(y_test, y_pred, zero_division=0),
        "f1": f1_score(y_test, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_test, y_proba),
        "pr_auc": average_precision_score(y_test, y_proba)
    }

    print("\n" + "=" * 60)
    print(f" {dataset_name.upper()} SET EVALUATION")
    print("=" * 60)
    for k, v in metrics.items():

```



```

        print(f" {k.capitalize():<10}: {v:.4f}")

    print(f"\n {dataset_name} Classification Report:\n",
    ↪classification_report(y_test, y_pred, digits=3))
    print(f" {dataset_name} Confusion Matrix:\n", confusion_matrix(y_test,
    ↪y_pred))

    return y_pred, y_proba, metrics

def create_visualizations(y_test, y_proba, model_type, output_dir, prefix,
    ↪cv_metrics=None):
    os.makedirs(output_dir, exist_ok=True)

    # 1. ROC & PR Curves
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

    fpr, tpr, _ = roc_curve(y_test, y_proba)
    roc_auc = roc_auc_score(y_test, y_proba)
    ax1.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}", linewidth=2, color="C0")
    ax1.plot([0, 1], [0, 1], "k--", alpha=0.5)
    ax1.set_title(f"ROC Curve ({model_type.upper()})")
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    prec, rec, _ = precision_recall_curve(y_test, y_proba)
    pr_auc = average_precision_score(y_test, y_proba)
    ax2.plot(rec, prec, label=f"AP = {pr_auc:.3f}", linewidth=2, color="C1")
    ax2.set_title(f"Precision-Recall Curve ({model_type.upper()})")
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(f"{output_dir}/{prefix}_roc_pr_curves.png", dpi=200)
    plt.show()

    # 2. Confusion Matrix
    y_pred = (y_proba >= 0.5).astype(int)
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Pred 0",
    ↪"Pred 1"], yticklabels=["True 0", "True 1"])
    plt.title(f"Confusion Matrix ({model_type.upper()})")
    plt.savefig(f"{output_dir}/{prefix}_confusion_matrix.png", dpi=200)
    plt.show()

    # 3. CV Comparison
    if cv_metrics:

```

```

names = list(cv_metrics.keys())
means = [cv_metrics[m]["test_mean"] for m in names]
stds = [cv_metrics[m]["test_std"] for m in names]

plt.figure(figsize=(10, 5))
bars = plt.bar(names, means, yerr=stds, capsize=5, alpha=0.7)
for i, b in enumerate(bars): b.set_color(f"C{i}")
plt.title(f"Cross-Validation Performance ({model_type.upper()})")
plt.ylim(0, 1)
plt.grid(axis='y', alpha=0.3)
plt.savefig(f"{output_dir}/{prefix}_cv_results.png", dpi=200)
plt.show()

# =====
# 8) SAVING RESULTS
# =====
def save_results(model, metrics, df_clean, target_col, feature_names,
    ↪output_dir, prefix, cv_metrics=None):
    print(f"\n SAVING OUTPUTS to {output_dir}/ ...")

    # 1. Metrics Text File
    with open(f"{output_dir}/{prefix}_metrics.txt", "w") as f:
        f.write(f"MODEL RESULTS: {prefix.upper()}\n{'='*40}\n")
        f.write("TEST SET METRICS:\n")
        for k, v in metrics.items(): f.write(f"{k}: {v:.4f}\n")

        if cv_metrics:
            f.write("\nCROSS-VALIDATION:\n")
            for k, v in cv_metrics.items():
                f.write(f"{k}: {v['test_mean']:.4f} ± {v['test_std']:.4f}\n")

    # 2. Excel Predictions
    X_full = df_clean.drop(columns=[target_col], errors="ignore").
    ↪reindex(columns=feature_names)
    preds = model.predict_proba(X_full)[: , 1]

    out_df = df_clean.copy()
    out_df[f"{prefix}_Prob"] = preds
    out_df[f"{prefix}_Pred"] = (preds >= 0.5).astype(int)
    out_df.to_excel(f"{output_dir}/{prefix}_predictions.xlsx", index=False)

    # 3. Model
    joblib.dump(model, f"{output_dir}/{prefix}_model.joblib")
    print(" All files saved successfully.")

# =====
# 9) MAIN PIPELINE

```

```

# =====
def run_comprehensive_model_analysis(target_bf="BF1", model_type="svm",
    ↪df=None):
    """
    Run pipeline for SVM or KNN.
    model_type: 'svm' or 'knn'
    """
    try:
        model_type = model_type.lower()
        prefix = f"{target_bf}_{model_type}"
        output_dir = f"{prefix}_outputs"

        print(f" STARTING ANALYSIS: {model_type.upper()} on {target_bf}")

        if df is None:
            try: df = load_excel_from_colab()
            except: df = load_excel_locally()

        # Target Setup
        target_col = detect_bf_target(df, target_bf)
        y, valid_mask = preprocess_bf_target(df[target_col], target_col)
        df_clean = df.loc[valid_mask].copy()

        # Features
        X_final, feature_names = build_feature_matrix(df_clean, target_col)

        # 1. CV
        cv_metrics, _ = perform_cross_validation(X_final, y, model_type,
    ↪cv_folds=10)

        # 2. Train
        model, X_train, X_test, X_val, y_train, y_test, y_val =
    ↪train_model_comprehensive(
            X_final, y, model_type, use_validation_set=True
        )

        # 3. Eval
        _, y_proba, test_metrics = evaluate_model_metrics(model, X_test,
    ↪y_test, "Test")

        # 4. Viz
        create_visualizations(y_test, y_proba, model_type, output_dir, prefix,
    ↪cv_metrics)

        # 5. Save
        save_results(model, test_metrics, df_clean, target_col, feature_names,
    ↪output_dir, prefix, cv_metrics)

```

```

        print("\n DONE.")
        return model

    except Exception as e:
        print(f"\n Error: {e}")
        return None

# ===== USAGE =====
if __name__ == "__main__":
    # Example 1: Run SVM on BF6 (Default)
    run_comprehensive_model_analysis(target_bf="BF6", model_type="svm")

    # Example 2: Run KNN (Uncomment to use)
    # run_comprehensive_model_analysis(target_bf="BF6", model_type="knn")

```

STARTING ANALYSIS: SVM on BF6

Please select the Excel file with your encoded dataset...

<IPython.core.display.HTML object>

Saving rf_dataset_encoded_dataset.xlsx to rf_dataset_encoded_dataset (5).xlsx

Loaded dataset: (1002, 85) (rows, cols)

BF6 column not found automatically.

Available columns: ['D', 'AY', 'AZ', 'BA', 'BB', 'BC', 'BD', 'BE_01', 'BE_02', 'BE_03', 'BE_04', 'BE_05', 'BE_06', 'BE_07', 'BE_08', 'BE_09', 'BE_10', 'BF', 'BG_01', 'BG_02', 'BG_03', 'BG_04', 'BG_05', 'BG_06', 'BH', 'BI', 'BN', 'BO', 'BP', 'BQ', 'BR', 'BS', 'BT', 'BU', 'BV', 'BW_01', 'BW_02', 'BW_03', 'BW_04', 'BW_05', 'BX', 'BY', 'CB', 'CD', 'CE', 'CF', 'CG', 'CH', 'CI', 'CK', 'CL', 'CM_01', 'CM_02', 'CM_03', 'CM_04', 'CM_05', 'CM_06', 'CN', 'CO_01', 'CO_02', 'CO_03', 'CO_04', 'CO_05', 'CO_06', 'CO_07', 'CO_08', 'CO_09', 'CP', 'CR_01', 'CR_02', 'CR_03', 'CR_04', 'CS', 'CT_01', 'CT_02', 'CT_03', 'CT_04', 'CT_05', 'CT_06', 'CU', 'Occupation', 'Financial Behavior', 'Family', 'Lifestyle', 'Digital Literacy']

Enter exact column name for BF6: BG_06

TARGET ANALYSIS: BG_06

Value distribution:

BG_06

0 783

1 219

Name: count, dtype: int64

Final class balance: {0: 0.781, 1: 0.219}

FEATURE ENGINEERING

Final feature matrix: (1002, 84) (rows, cols)

PERFORMING 10-FOLD CROSS-VALIDATION (SVM)

Configured SVM (RBF Kernel, C=1.0, Probability=True)
CROSS-VALIDATION RESULTS (Mean $\pm$ Std):

ACCURACY : 0.8772 $\pm$ 0.0262
PRECISION : 0.7053 $\pm$ 0.0485
RECALL : 0.7433 $\pm$ 0.1175
F1 : 0.7214 $\pm$ 0.0771
ROC_AUC : 0.9161 $\pm$ 0.0373
AVERAGE_PRECISION: 0.8167 $\pm$ 0.0919

MODEL TRAINING: SVM

CREATING SPLITS (Val=0.2)

Configured SVM (RBF Kernel, C=1.0, Probability=True)
Fitting model...

TEST SET EVALUATION

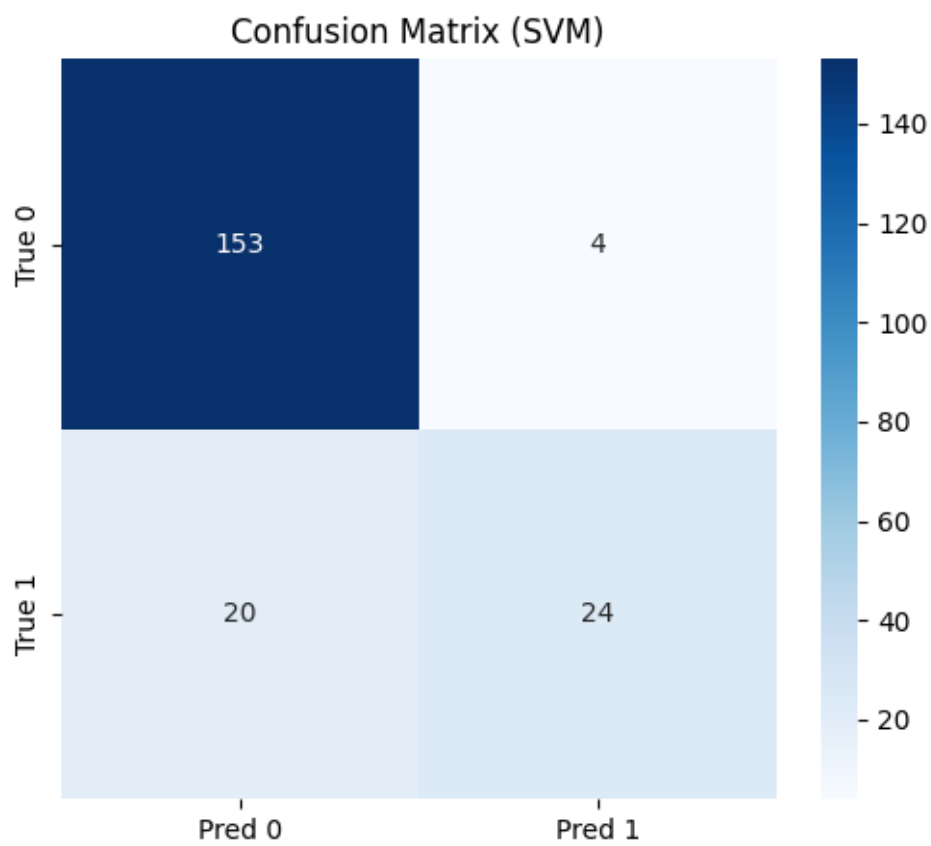
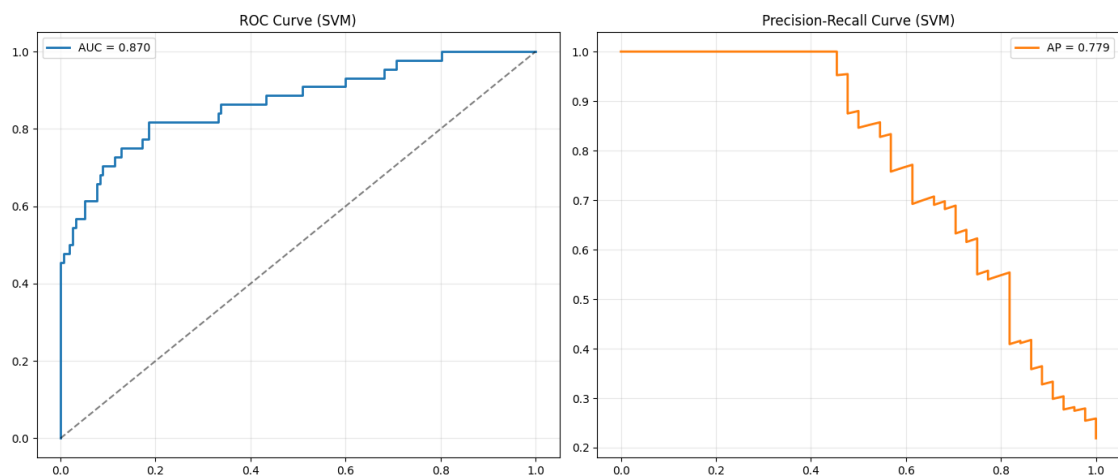
Accuracy : 0.8607
Precision : 0.6905
Recall : 0.6591
F1 : 0.6744
Roc_auc : 0.8700
Pr_auc : 0.7791

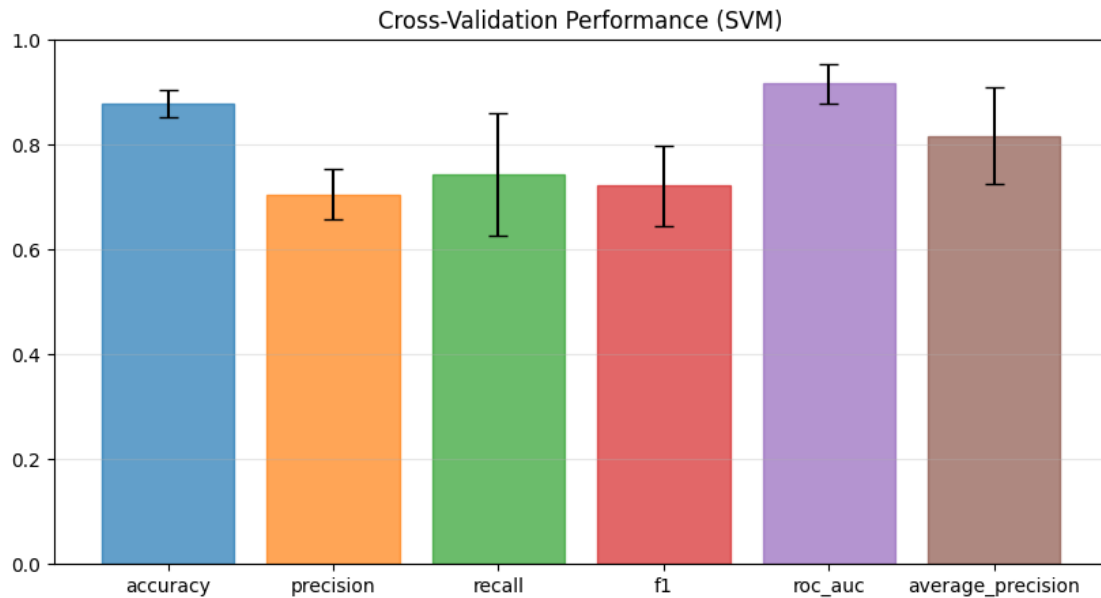
Test Classification Report:

	precision	recall	f1-score	support
0	0.906	0.917	0.911	157
1	0.690	0.659	0.674	44
accuracy			0.861	201
macro avg	0.798	0.788	0.793	201
weighted avg	0.859	0.861	0.860	201

Test Confusion Matrix:

[[144 13]
[15 29]]





SAVING OUTPUTS to BF6_svm_outputs/ ...
All files saved successfully.

DONE.

```
[1]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
[ ]: # Install the package for Tex and then convert to PDF directly as LaTeX
!sudo apt-get install texlive-xetex texlive-fonts-recommended
    ↳ texlive-plain-generic pandoc
# Provide the file path of the notebook file inside the quotations
!jupyter nbconvert --to pdf "/content/drive/MyDrive/Colab Notebooks/CSE thesis_
    ↳ SVM P3.ipynb"
```

Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
pandoc is already the newest version (2.9.2.1-3ubuntu2).
pandoc set to manually installed.
The following additional packages will be installed:
dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12

libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
libsynctex2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzip-0-13
lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils
teckit tex-common tex-gyre texlive-base texlive-binaries texlive-latex-base
texlive-latex-extra texlive-latex-recommended texlive-pictures tipa
xfonts-encodings xfonts-utils

Suggested packages:

fonts-noto fonts-freefont-otf | fonts-freefont-ttf libavalon-framework-java
libcommons-logging-java-doc libexcalibur-logkit-java liblog4j1.2-java
poppler-utils ghostscript fonts-japanese-mincho | fonts-ipafont-mincho
fonts-japanese-gothic | fonts-ipafont-gothic fonts-arphic-ukai
fonts-arphic-uming fonts-nanum ri ruby-dev bundler debhelper gv
| postscript-viewer perl-tk xpdf | pdf-viewer xzdec
texlive-fonts-recommended-doc texlive-latex-base-doc python3-pygments
icc-profiles libfile-which-perl libspreadsheet-parseexcel-perl
texlive-latex-extra-doc texlive-latex-recommended-doc texlive-luatex
texlive-pstricks dot2tex prerex texlive-pictures-doc vprerex
default-jre-headless tipa-doc

The following NEW packages will be installed:

dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
libsynctex2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzip-0-13
lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils
teckit tex-common tex-gyre texlive-base texlive-binaries
texlive-fonts-recommended texlive-latex-base texlive-latex-extra
texlive-latex-recommended texlive-pictures texlive-plain-generic
texlive-xetex tipa xfonts-encodings xfonts-utils

0 upgraded, 53 newly installed, 0 to remove and 2 not upgraded.

Need to get 182 MB of archives.

After this operation, 571 MB of additional disk space will be used.

Get:1 <http://archive.ubuntu.com/ubuntu/jammy/main amd64 fonts-droid-fallback all 1:6.0.1r16-1.1build1> [1,805 kB]

Get:2 <http://archive.ubuntu.com/ubuntu/jammy/main amd64 fonts-lato all 2.0-2.1> [2,696 kB]

Get:3 <http://archive.ubuntu.com/ubuntu/jammy/main amd64 poppler-data all 0.4.11-1> [2,171 kB]

Get:4 <http://archive.ubuntu.com/ubuntu/jammy/universe amd64 tex-common all 6.17> [33.7 kB]

Get:5 <http://archive.ubuntu.com/ubuntu/jammy/main amd64 fonts-urw-base35 all 20200910-1> [6,367 kB]

Get:6 <http://archive.ubuntu.com/ubuntu/jammy-updates/main amd64 libgs9-common all 9.55.0-dfsg1-0ubuntu5.13> [753 kB]

Get:7 <http://archive.ubuntu.com/ubuntu/jammy-updates/main amd64 libidn12 amd64 1.38-4ubuntu1> [60.0 kB]

Get:8 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libijs-0.35 amd64 0.35-15build2 [16.5 kB]
Get:9 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libjbig2dec0 amd64 0.19-3build2 [64.7 kB]
Get:10 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libgs9 amd64 9.55.0~dfsg1-0ubuntu5.13 [5,032 kB]
Get:11 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libkpathsea6 amd64 2021.20210626.59705-1ubuntu0.2 [60.4 kB]
Get:12 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libwoff1 amd64 1.0.2-1build4 [45.2 kB]
Get:13 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 dvisvgm amd64 2.13.1-1 [1,221 kB]
Get:14 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 fonts-lmodern all 2.004.5-6.1 [4,532 kB]
Get:15 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-noto-mono all 20201225-1build1 [397 kB]
Get:16 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 fonts-texgyre all 20180621-3.1 [10.2 MB]
Get:17 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libapache-pom-java all 18-1 [4,720 B]
Get:18 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libcommons-parent-java all 43-1 [10.8 kB]
Get:19 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libcommons-logging-java all 1.2-2 [60.3 kB]
Get:20 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libptexenc1 amd64 2021.20210626.59705-1ubuntu0.2 [39.1 kB]
Get:21 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rubygems-integration all 1.18 [5,336 B]
Get:22 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby3.0 amd64 3.0.2-7ubuntu2.11 [50.1 kB]
Get:23 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-rubygems all 3.3.5-2ubuntu1.2 [228 kB]
Get:24 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby amd64 1:3.0~exp1 [5,100 B]
Get:25 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rake all 13.0.6-2 [61.7 kB]
Get:26 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby-net-telnet all 0.1.1-2 [12.6 kB]
Get:27 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-webrick all 1.7.0-3ubuntu0.2 [52.5 kB]
Get:28 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-xmlrpc all 0.3.2-1ubuntu0.1 [24.9 kB]
Get:29 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libruby3.0 amd64 3.0.2-7ubuntu2.11 [5,114 kB]
Get:30 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libsynchronet2 amd64 2021.20210626.59705-1ubuntu0.2 [55.6 kB]
Get:31 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libteckit0 amd64 2.5.11+ds1-1 [421 kB]

```

Get:32 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libtexlua53
amd64 2021.20210626.59705-1ubuntu0.2 [120 kB]
Get:33 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libtexluajit2
amd64 2021.20210626.59705-1ubuntu0.2 [267 kB]
Get:34 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libzip-0-13 amd64
0.13.72+dfsg.1-1.1 [27.0 kB]
Get:35 http://archive.ubuntu.com/ubuntu jammy/main amd64 xfonts-encodings all
1:1.0.5-0ubuntu2 [578 kB]
Get:36 http://archive.ubuntu.com/ubuntu jammy/main amd64 xfonts-utils amd64
1:7.7+6build2 [94.6 kB]
Get:37 http://archive.ubuntu.com/ubuntu jammy/universe amd64 lmodern all
2.004.5-6.1 [9,471 kB]
Get:38 http://archive.ubuntu.com/ubuntu jammy/universe amd64 preview-latex-style
all 12.2-1ubuntu1 [185 kB]
Get:39 http://archive.ubuntu.com/ubuntu jammy/main amd64 t1utils amd64
1.41-4build2 [61.3 kB]
Get:40 http://archive.ubuntu.com/ubuntu jammy/universe amd64 teckit amd64
2.5.11+ds1-1 [699 kB]
Get:41 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tex-gyre all
20180621-3.1 [6,209 kB]
Get:42 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 texlive-
binaries amd64 2021.20210626.59705-1ubuntu0.2 [9,860 kB]
Get:43 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-base all
2021.20220204-1 [21.0 MB]
Get:44 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-fonts-
recommended all 2021.20220204-1 [4,972 kB]
Get:45 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-base
all 2021.20220204-1 [1,128 kB]
Get:46 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libfontbox-java all
1:1.8.16-2 [207 kB]
Get:47 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libpdfbox-java all
1:1.8.16-2 [5,199 kB]
Get:48 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-
recommended all 2021.20220204-1 [14.4 MB]
Get:49 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-pictures
all 2021.20220204-1 [8,720 kB]
Get:50 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-extra
all 2021.20220204-1 [13.9 MB]
Get:51 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-plain-
generic all 2021.20220204-1 [27.5 MB]
Get:52 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tipa all 2:1.3-21
[2,967 kB]
Get:53 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-xetex all
2021.20220204-1 [12.4 MB]
Fetched 182 MB in 7s (26.4 MB/s)
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based
frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 78,

```

```

<> line 53.)
debconf: falling back to frontend: Readline
debconf: unable to initialize frontend: Readline
debconf: (This frontend requires a controlling tty.)
debconf: falling back to frontend: Teletype
dpkg-preconfigure: unable to re-open stdin:
Selecting previously unselected package fonts-droid-fallback.
(Reading database ... 117540 files and directories currently installed.)
Preparing to unpack .../00-fonts-droid-fallback_1%3a6.0.1r16-1.1build1_all.deb
...
Unpacking fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Selecting previously unselected package fonts-lato.
Preparing to unpack .../01-fonts-lato_2.0-2.1_all.deb ...
Unpacking fonts-lato (2.0-2.1) ...
Selecting previously unselected package poppler-data.
Preparing to unpack .../02-poppler-data_0.4.11-1_all.deb ...
Unpacking poppler-data (0.4.11-1) ...
Selecting previously unselected package tex-common.
Preparing to unpack .../03-tex-common_6.17_all.deb ...
Unpacking tex-common (6.17) ...
Selecting previously unselected package fonts-urw-base35.
Preparing to unpack .../04-fonts-urw-base35_20200910-1_all.deb ...
Unpacking fonts-urw-base35 (20200910-1) ...
Selecting previously unselected package libgs9-common.
Preparing to unpack .../05-libgs9-common_9.55.0~dfsg1-0ubuntu5.13_all.deb ...
Unpacking libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libidn12:amd64.
Preparing to unpack .../06-libidn12_1.38-4ubuntu1_amd64.deb ...
Unpacking libidn12:amd64 (1.38-4ubuntu1) ...
Selecting previously unselected package libijs-0.35:amd64.
Preparing to unpack .../07-libijs-0.35_0.35-15build2_amd64.deb ...
Unpacking libijs-0.35:amd64 (0.35-15build2) ...
Selecting previously unselected package libjbig2dec0:amd64.
Preparing to unpack .../08-libjbig2dec0_0.19-3build2_amd64.deb ...
Unpacking libjbig2dec0:amd64 (0.19-3build2) ...
Selecting previously unselected package libgs9:amd64.
Preparing to unpack .../09-libgs9_9.55.0~dfsg1-0ubuntu5.13_amd64.deb ...
Unpacking libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libkpathsea6:amd64.
Preparing to unpack .../10-libkpathsea6_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libwoff1:amd64.
Preparing to unpack .../11-libwoff1_1.0.2-1build4_amd64.deb ...
Unpacking libwoff1:amd64 (1.0.2-1build4) ...
Selecting previously unselected package dvisvgm.
Preparing to unpack .../12-dvisvgm_2.13.1-1_amd64.deb ...
Unpacking dvisvgm (2.13.1-1) ...

```

```

Selecting previously unselected package fonts-lmodern.
Preparing to unpack .../13-fonts-lmodern_2.004.5-6.1_all.deb ...
Unpacking fonts-lmodern (2.004.5-6.1) ...
Selecting previously unselected package fonts-noto-mono.
Preparing to unpack .../14-fonts-noto-mono_20201225-1build1_all.deb ...
Unpacking fonts-noto-mono (20201225-1build1) ...
Selecting previously unselected package fonts-texgyre.
Preparing to unpack .../15-fonts-texgyre_20180621-3.1_all.deb ...
Unpacking fonts-texgyre (20180621-3.1) ...
Selecting previously unselected package libapache-pom-java.
Preparing to unpack .../16-libapache-pom-java_18-1_all.deb ...
Unpacking libapache-pom-java (18-1) ...
Selecting previously unselected package libcommons-parent-java.
Preparing to unpack .../17-libcommons-parent-java_43-1_all.deb ...
Unpacking libcommons-parent-java (43-1) ...
Selecting previously unselected package libcommons-logging-java.
Preparing to unpack .../18-libcommons-logging-java_1.2-2_all.deb ...
Unpacking libcommons-logging-java (1.2-2) ...
Selecting previously unselected package libptexenc1:amd64.
Preparing to unpack .../19-libptexenc1_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package rubygems-integration.
Preparing to unpack .../20-rubygems-integration_1.18_all.deb ...
Unpacking rubygems-integration (1.18) ...
Selecting previously unselected package ruby3.0.
Preparing to unpack .../21-ruby3.0_3.0.2-7ubuntu2.11_amd64.deb ...
Unpacking ruby3.0 (3.0.2-7ubuntu2.11) ...
Selecting previously unselected package ruby-rubygems.
Preparing to unpack .../22-ruby-rubygems_3.3.5-2ubuntu1.2_all.deb ...
Unpacking ruby-rubygems (3.3.5-2ubuntu1.2) ...
Selecting previously unselected package ruby.
Preparing to unpack .../23-ruby_1%3a3.0~exp1_amd64.deb ...
Unpacking ruby (1:3.0~exp1) ...
Selecting previously unselected package rake.
Preparing to unpack .../24-rake_13.0.6-2_all.deb ...
Unpacking rake (13.0.6-2) ...
Selecting previously unselected package ruby-net-telnet.
Preparing to unpack .../25-ruby-net-telnet_0.1.1-2_all.deb ...
Unpacking ruby-net-telnet (0.1.1-2) ...
Selecting previously unselected package ruby-webrick.
Preparing to unpack .../26-ruby-webrick_1.7.0-3ubuntu0.2_all.deb ...
Unpacking ruby-webrick (1.7.0-3ubuntu0.2) ...
Selecting previously unselected package ruby-xmlrpc.
Preparing to unpack .../27-ruby-xmlrpc_0.3.2-1ubuntu0.1_all.deb ...
Unpacking ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Selecting previously unselected package libruby3.0:amd64.
Preparing to unpack .../28-libruby3.0_3.0.2-7ubuntu2.11_amd64.deb ...

```

```

Unpacking libruby3.0:amd64 (3.0.2-7ubuntu2.11) ...
Selecting previously unselected package libsyntax2:amd64.
Preparing to unpack .../29-libsyntax2_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libsyntax2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libteckit0:amd64.
Preparing to unpack .../30-libteckit0_2.5.11+ds1-1_amd64.deb ...
Unpacking libteckit0:amd64 (2.5.11+ds1-1) ...
Selecting previously unselected package libtexlua53:amd64.
Preparing to unpack .../31-libtexlua53_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libtexluajit2:amd64.
Preparing to unpack
.../32-libtexluajit2_2021.20210626.59705-1ubuntu0.2_amd64.deb ...
Unpacking libtexluajit2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libzzip-0-13:amd64.
Preparing to unpack .../33-libzzip-0-13_0.13.72+dfsg.1-1.1_amd64.deb ...
Unpacking libzzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Selecting previously unselected package xfonts-encodings.
Preparing to unpack .../34-xfonts-encodings_1%3a1.0.5-0ubuntu2_all.deb ...
Unpacking xfonts-encodings (1:1.0.5-0ubuntu2) ...
Selecting previously unselected package xfonts-utils.
Preparing to unpack .../35-xfonts-utils_1%3a7.7+6build2_amd64.deb ...
Unpacking xfonts-utils (1:7.7+6build2) ...
Selecting previously unselected package lmodern.
Preparing to unpack .../36-lmodern_2.004.5-6.1_all.deb ...
Unpacking lmodern (2.004.5-6.1) ...
Selecting previously unselected package preview-latex-style.
Preparing to unpack .../37-preview-latex-style_12.2-1ubuntu1_all.deb ...
Unpacking preview-latex-style (12.2-1ubuntu1) ...
Selecting previously unselected package tlutils.
Preparing to unpack .../38-tlutils_1.41-4build2_amd64.deb ...
Unpacking tlutils (1.41-4build2) ...
Selecting previously unselected package teckit.
Preparing to unpack .../39-teckit_2.5.11+ds1-1_amd64.deb ...
Unpacking teckit (2.5.11+ds1-1) ...
Selecting previously unselected package tex-gyre.
Preparing to unpack .../40-tex-gyre_20180621-3.1_all.deb ...
Unpacking tex-gyre (20180621-3.1) ...
Selecting previously unselected package texlive-binaries.
Preparing to unpack .../41-texlive-
binaries_2021.20210626.59705-1ubuntu0.2_amd64.deb ...
Unpacking texlive-binaries (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package texlive-base.
Preparing to unpack .../42-texlive-base_2021.20220204-1_all.deb ...
Unpacking texlive-base (2021.20220204-1) ...
Selecting previously unselected package texlive-fonts-recommended.

```

```

Preparing to unpack .../43-texlive-fonts-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-fonts-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-base.
Preparing to unpack .../44-texlive-latex-base_2021.20220204-1_all.deb ...
Unpacking texlive-latex-base (2021.20220204-1) ...
Selecting previously unselected package libfontbox-java.
Preparing to unpack .../45-libfontbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libfontbox-java (1:1.8.16-2) ...
Selecting previously unselected package libpdfbox-java.
Preparing to unpack .../46-libpdfbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libpdfbox-java (1:1.8.16-2) ...
Selecting previously unselected package texlive-latex-recommended.
Preparing to unpack .../47-texlive-latex-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-latex-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive-pictures.
Preparing to unpack .../48-texlive-pictures_2021.20220204-1_all.deb ...
Unpacking texlive-pictures (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-extra.
Preparing to unpack .../49-texlive-latex-extra_2021.20220204-1_all.deb ...
Unpacking texlive-latex-extra (2021.20220204-1) ...
Selecting previously unselected package texlive-plain-generic.
Preparing to unpack .../50-texlive-plain-generic_2021.20220204-1_all.deb ...
Unpacking texlive-plain-generic (2021.20220204-1) ...
Selecting previously unselected package tipa.
Preparing to unpack .../51-tipa_2%3a1.3-21_all.deb ...
Unpacking tipa (2:1.3-21) ...
Selecting previously unselected package texlive-xetex.
Preparing to unpack .../52-texlive-xetex_2021.20220204-1_all.deb ...
Unpacking texlive-xetex (2021.20220204-1) ...
Setting up fonts-lato (2.0-2.1) ...
Setting up fonts-noto-mono (20201225-1build1) ...
Setting up libwoff1:amd64 (1.0.2-1build4) ...
Setting up libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up libijs-0.35:amd64 (0.35-15build2) ...
Setting up libtexluaajit2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up libfontbox-java (1:1.8.16-2) ...
Setting up rubygems-integration (1.18) ...
Setting up libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Setting up fonts-urw-base35 (20200910-1) ...
Setting up poppler-data (0.4.11-1) ...
Setting up tex-common (6.17) ...
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based
frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line
78.)
debconf: falling back to frontend: Readline
update-language: texlive-base not installed and configured, doing nothing!
Setting up libjbig2dec0:amd64 (0.19-3build2) ...

```

```

Setting up libteckit0:amd64 (2.5.11+ds1-1) ...
Setting up libapache-pom-java (18-1) ...
Setting up ruby-net-telnet (0.1.1-2) ...
Setting up xfonts-encodings (1:1.0.5-0ubuntu2) ...
Setting up t1utils (1.41-4build2) ...
Setting up libidn12:amd64 (1.38-4ubuntu1) ...
Setting up fonts-texgyre (20180621-3.1) ...
Setting up libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up ruby-webrick (1.7.0-3ubuntu0.2) ...
Setting up fonts-lmodern (2.004.5-6.1) ...
Setting up fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Setting up ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Setting up libsyntax2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Setting up teckit (2.5.11+ds1-1) ...
Setting up libpdfbox-java (1:1.8.16-2) ...
Setting up libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Setting up preview-latex-style (12.2-1ubuntu1) ...
Setting up libcommons-parent-java (43-1) ...
Setting up dvisvgm (2.13.1-1) ...
Setting up libcommons-logging-java (1.2-2) ...
Setting up xfonts-utils (1:7.7+6build2) ...
Setting up libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up texlive-binaries (2021.20210626.59705-1ubuntu0.2) ...
update-alternatives: using /usr/bin/xdvi-xaw to provide /usr/bin/xdvi.bin
(xdvi.bin) in auto mode
update-alternatives: using /usr/bin/bibtex.original to provide /usr/bin/bibtex
(bibtex) in auto mode
Setting up lmodern (2.004.5-6.1) ...
Setting up texlive-base (2021.20220204-1) ...
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
mktexlsr: Updating /var/lib/texmf/ls-R-TEXLIVEDIST...
mktexlsr: Updating /var/lib/texmf/ls-R-TEXMFMAIN...
mktexlsr: Updating /var/lib/texmf/ls-R...
mktexlsr: Done.
tl-paper: setting paper size for dvips to a4:
/var/lib/texmf/dvips/config/config-paper.ps
tl-paper: setting paper size for dvipdfmx to a4:
/var/lib/texmf/dvipdfmx/dvipdfmx-paper.cfg
tl-paper: setting paper size for xdvi to a4: /var/lib/texmf/xdvi/XDvi-paper
tl-paper: setting paper size for pdftex to a4: /var/lib/texmf/tex/generic/tex-
ini-files/pdftexconfig.tex
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based
frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line

```

```

78.)
debconf: falling back to frontend: Readline
Setting up tex-gyre (20180621-3.1) ...
Setting up texlive-plain-generic (2021.20220204-1) ...
Setting up texlive-latex-base (2021.20220204-1) ...
Setting up texlive-latex-recommended (2021.20220204-1) ...
Setting up texlive-pictures (2021.20220204-1) ...
Setting up texlive-fonts-recommended (2021.20220204-1) ...
Setting up tipa (2:1.3-21) ...
Setting up texlive-latex-extra (2021.20220204-1) ...
Setting up texlive-xetex (2021.20220204-1) ...
Setting up rake (13.0.6-2) ...
Setting up libruby3.0:amd64 (3.0.2-7ubuntu2.11) ...
Setting up ruby3.0 (3.0.2-7ubuntu2.11) ...
Setting up ruby (1:3.0~exp1) ...
Setting up ruby-rubygems (3.3.5-2ubuntu1.2) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for mailcap (3.70+nmu1ubuntu1) ...
Processing triggers for fontconfig (2.13.1-4.2ubuntu5) ...
Processing triggers for libc-bin (2.35-0ubuntu3.11) ...
/sbin/ldconfig.real: /usr/local/lib/libtcm.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_0.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero_v2.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_opencl.so.0 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libumf.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc_proxy.so.2 is not a symbolic

```


link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc.so.2 is not a symbolic link

Processing triggers for tex-common (6.17) ...

debconf: unable to initialize frontend: Dialog

debconf: (No usable dialog-like program is installed, so the dialog based
frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line
78.)

debconf: falling back to frontend: Readline

Running updmap-sys. This may take some time... done.

Running mktexlsr /var/lib/texmf ... done.

Building format(s) --all.

 This may take some time...